CustardFin

A Cost-Optimized Hybrid Edge-Cloud Media Streaming and Community Discovery Platform

Cloud Computing Project Report

Submitted in Partial Fulfillment of the Requirements for

Master of Computer Applications (MCA)

Cloud Computing Laboratory

Submitted by:

Anish Ghadi	**Seat No.:** 25P0320009
Anosh Chodankar	**Seat No.:** 25P0320010

Academic Year 2026–2027

Department of Computer Applications

University / College of Higher Education

Candidate Declaration

We hereby declare that the project entitled **"CustardFin: A Cost-Optimized Hybrid Edge-Cloud Media Streaming and Community Discovery Platform"** submitted by us in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) is our own original work. All sources of assistance, libraries, frameworks, and literature utilized have been properly cited and acknowledged.

Anish Ghadi
Seat No.: 25P0320009

Anosh Chodankar
Seat No.: 25P0320010

Date: November 2026
Place: Goa, India

Abstract

Traditional personal media streaming architectures present an intractable trade-off: self-hosted media servers (such as Jellyfin and Plex) trap content on private home local networks behind residential ISP Carrier-Grade Network Address Translation (CGNAT), whereas centralized public video hosting platforms (such as YouTube and AWS S3/CloudFront) impose recurring cloud storage bills, bandwidth egress penalties, and complete loss of video ownership.

CustardFin solves this challenge by introducing a hybrid edge-cloud video streaming architecture adhering to a strict governing invariant: *video binaries remain permanently on the content owner's home edge machine, while the cloud control tier manages only a lightweight searchable catalog and ephemeral stream coordination.* Marking a video "Public" syncs only a ≤ 3 KB JSON metadata manifest to a cloud PostgreSQL database without uploading the underlying media binary, achieving approximately **98% reduction in cloud operational costs** compared to conventional cloud VOD infrastructure.

The platform implements pre-encoded 4-second HTTP Live Streaming (HLS) with automatic hardware acceleration probing (NVIDIA NVENC with automatic CPU fallback), reducing playback compute to under 3% CPU utilization. Home edge nodes punch through residential ISP CGNAT via outbound-only Transport Layer Security (TLS) reverse tunnels using Fast Reverse Proxy (FRP), eliminating inbound open router port requirements. To protect constrained residential uplink bandwidth, a cloud-side 24-hour LRU *Origin Shield* object cache serves subsequent viewers after an initial on-demand pull, complemented by dynamic master playlist rewriting (adaptive ladder pruning) and a strict 10-stream remote admission limit. Furthermore, local clients benefit from zero-latency LAN streaming via an embedded, offline-capable Mini-Jellyfin web interface on the home daemon. The entire system is implemented in Go and Next.js utilizing 100% free and open-source software (FOSS), with multi-topology deployment verified live across local development containers, Floci cloud emulation, and Google Cloud Platform (GCP).

Contents

Candidate Declaration i

Abstract ii

1 Introduction and Problem Statement 1
- 1.1 Overview and Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Project Objectives and Target Metrics . . . 3
- 1.4 Non-Negotiable Invariants . . . 3
- 1.5 Scope of the Project . . . 5
  - 1.5.1 In-Scope System Boundaries . . . 5
  - 1.5.2 Out-of-Scope (Version 1 Boundaries) . . . 5
- 1.6 Organization of the Report . . . 6

2 Literature Survey and Background Study 7
- 2.1 Analysis of Centralized Cloud Video Platforms . . . 7
  - 2.1.1 Architectural Pipeline . . . 7
  - 2.1.2 Economic and Operational Inefficiencies . . . 8
- 2.2 Analysis of Self-Hosted Home Media Servers . . . 8
  - 2.2.1 Architecture and Local Network Strengths . . . 8
  - 2.2.2 Systemic Remote Access and Scaling Vulnerabilities . . . 9
- 2.3 Hybrid and Peer-to-Peer Streaming Approaches . . . 10
  - 2.3.1 PeerTube and ActivityPub Federation . . . 10
  - 2.3.2 Fundamental Limitations of P2P Streaming . . . 10
- 2.4 Comparative Feature and Gap Analysis . . . 11
  - 2.4.1 Identified Research and Engineering Gap . . . 11
- 2.5 Theoretical Foundations: Video Encoding, HLS, and Tunneling . . . 11
  - 2.5.1 Digital Video Compression and the H.264 / AVC Standard . . . 13
  - 2.5.2 Hardware-Accelerated Encoding: NVIDIA NVENC . . . 13
  - 2.5.3 HTTP Live Streaming (RFC 8216) and Keyframe Alignment . . . 14
  - 2.5.4 Rationale for 4-Second Segment Granularity . . . 14
  - 2.5.5 Reverse Proxy Tunneling Architecture . . . 14

3 System Architecture and Design 16
- 3.1 Overall Two-Tier System Architecture . . . 16
- 3.2 Home Edge Node Architecture (`apps/edge`) . . . 17
  - 3.2.1 Subsystem Decomposition . . . 17

3.3 Cloud Control Plane Architecture (apps/api) . . . 18
3.3.1 Database and Cache Layer . . . 18
3.3.2 Origin Shield Pull-Through Cache . . . 18
3.3.3 Stream Router and Admission Limiter . . . 19
3.4 Presentation Tier Architecture (apps/web) . . . 19
3.4.1 Server vs. Client Component Segregation . . . 19
3.5 End-to-End Data Flows and Sequence Workflows . . . 20
3.5.1 Workflow 1: Local Ingestion and Pre-Encoding . . . 20
3.5.2 Workflow 2: Public Manifest Synchronization . . . 20
3.5.3 Workflow 3: Node Heartbeat and Liveness Tracking . . . 21
3.5.4 Workflow 4: Remote Playback Decision and Origin Shield Caching 21

4 Key Features and Technical Implementation 23
4.1 Local Media Organization and Private SQLite Storage (FR-1, FR-3) . . . 23
4.1.1 Recursive Ingestion Scanner . . . 23
4.1.2 Embedded Pure-Go SQLite Architecture . . . 23
4.2 Asynchronous Pre-Encoding and Hardware Acceleration (FR-4, FR-5) . . . 24
4.2.1 Adaptive Bitrate Ladder and Strict GOP Alignment . . . 24
4.2.2 Hardware Probe and Fault-Tolerant CPU Fallback . . . 25
4.3 Outbound Reverse Tunneling for CGNAT Bypass (FR-7) . . . 25
4.3.1 Tunnel Connection Mechanics . . . 25
4.4 Origin Shield Object Caching and URL Signing (FR-8, FR-10) . . . 26
4.4.1 Single-Upload Protection (Invariant 7) . . . 26
4.4.2 HMAC-SHA256 URL Signing . . . 26
4.5 Node Health and Ephemeral Heartbeat Liveness (FR-9) . . . 26
4.5.1 Redis TTL Beacons . . . 27
4.6 Dual-Path Streaming and Embedded Mini-Jellyfin (FR-6) . . . 27
4.6.1 Stream Router Proximity Resolution . . . 27
4.6.2 Embedded Mini-Jellyfin Standalone Hub . . . 28
4.7 User Accounts, Edge Node Claiming, and Video Management . . . 28
4.8 User Interface Walkthrough with Screenshots . . . 28
4.8.1 Community Catalog Feed . . . 28
4.8.2 Video Player with “Stats for Nerds” HUD . . . 29
4.8.3 My Edge Library Manager . . . 29
4.8.4 Embedded Standalone Mini-Jellyfin Hub . . . 29
4.8.5 Developer Settings and Diagnostics Modal . . . 29

5 Cost Analysis and Financial Feasibility 32
5.1 Methodology and Baseline Scenario . . . 32
5.2 Traditional Centralized Cloud Cost Model . . . 33
5.2.1 Component Breakdown . . . 33
5.2.2 Aggregate Centralized Burn Rate . . . 34
5.3 CustardFin Hybrid Architecture Cost Model . . . 34
5.3.1 Component Breakdown . . . 34
5.3.2 Aggregate CustardFin Burn Rate . . . 35
5.4 Comparative Cost Breakdown and ROI Analysis . . . 35
5.4.1 Mathematical Proof of Cost Reduction . . . 35
5.5 Sustainability and Scalability Impact . . . 37

6 Deployment, Testing, and Experimental Evaluation **38**
6.1 Multi-Topology Deployment Profiles . . . 38
6.2 Transcoding Performance Benchmarks (GPU vs CPU) . . . 39
6.2.1 Experimental Hardware Configuration . . . 39
6.2.2 Benchmark Results and Comparative Analysis . . . 39
6.2.3 Key Technical Insights . . . 39
6.3 Origin Shield Caching and 10-Viewer Load Simulation . . . 41
6.3.1 Experimental Setup . . . 41
6.3.2 Observed Cache Hit Progression . . . 41
6.3.3 Uplink Preservation Proof . . . 42
6.4 Admission Control and Capacity Limit Verification . . . 42
6.4.1 Test Scenario and Results . . . 42
6.5 Latency Analysis and Network Robustness . . . 43
6.5.1 Heartbeat Liveness Failover Test . . . 43
6.6 Automated Quality Gates and Test Coverage . . . 43

7 Team Contributions and Division of Work **45**
7.1 Introduction and Collaborative Model . . . 45
7.2 Contribution of Anish Ghadi . . . 46
7.3 Contribution of Anosh Chodankar . . . 47
7.4 Joint Responsibilities and Shared Milestones . . . 48
7.5 Summary Contribution Matrix . . . 49

8 Conclusion and Future Scope **51**
8.1 Summary of Achievements . . . 51
8.2 Limitations of the Current System (Version 1) . . . 52
8.3 Future Research and Engineering Enhancements . . . 53
8.4 Concluding Remarks . . . 53

List of Figures

2.1 Mechanics of Reverse Proxy NAT Traversal via Multiplexed Outbound Control Connections . . . 15

3.1 CustardFin Macro Two-Tier Distributed System Architecture Blueprint . 16
3.2 Local Ingestion, Hardware Encoding, and SQLite Indexing Sequence . . . 21
3.3 Ephemeral Heartbeat and Automated Node Liveness Detection 22
3.4 Dual-Path Playback Decision Tree and Origin Shield Single-Ingest Workflow 22

4.1 CustardFin Community Catalog Feed: Distributed Discovery and Real-Time Node Status . . . 29
4.2 Video Player Interface with Real-Time "Stats for Nerds" Engineering Telemetry HUD . . . 30
4.3 My Edge Library Modal: Node Management, Storage Quota, and Public/Private Toggles . . . 30
4.4 Standalone Embedded Mini-Jellyfin Offline Web Hub (Port 8089 Direct LAN Streaming) . . . 31
4.5 Developer Diagnostics and Grading Topology Configuration Interface . . 31

5.1 Itemized Operating Cost Comparison: Centralized Cloud VOD vs. CustardFin Hybrid Architecture . . . 36

6.1 Empirical Hardware Transcoding Benchmark: NVIDIA RTX 4060 NVENC vs. CPU Software . . . 40

List of Tables

1.1 CustardFin Core Objectives and Quantitative Targets 3

2.1 Comprehensive Architectural and Functional Comparison of Video Systems 12

5.1 Comprehensive Cost Comparison: Centralized Cloud VOD vs. CustardFin (100 Users, 5 TB Library, \$1 = ₹ 84.00) 36

6.1 Transcoding Performance Benchmark: NVIDIA NVENC vs. Multi-Threaded CPU 40
6.2 Origin Shield Cache Hit Progression Across 10 Concurrent Viewers . . . 41
6.3 End-to-End Playback Latency Across Operational Routing Modes 43

7.1 System Subsystem Responsibility and Workload Contribution Matrix . . 50

CHAPTER 1

Introduction and Problem Statement

1.1 Overview and Motivation

Over the past two decades, video streaming has emerged as the dominant consumer of global internet bandwidth, accounting for over 65% of all downstream network traffic. Modern media delivery is bifurcated into two mutually exclusive architectural paradigms, each presenting substantial structural trade-offs:

1. **Centralized Public Video Platforms (The YouTube / Cloud VOD Model):** Commercial cloud video-on-demand architectures rely on hyper-scale centralized infrastructure (e.g., AWS S3, Google Cloud Storage, Akamai CDN). Users upload raw video files to corporate servers, which handle automated ingestion, cloud-side distributed transcoding, catalog indexing, and global Content Delivery Network (CDN) replication. While this provides frictionless community discovery and massive scalability, it introduces heavy economic penalties. Cloud storage pricing (typically ₹ 1.932 / $0.023 per gigabyte per month on Amazon S3) combined with data egress transfer fees (₹ 7.56 / $0.09 per gigabyte) creates compounding monthly financial liabilities. Furthermore, content creators surrender custody of their original media files to platform algorithms, corporate content moderation, and intrusive advertisement monetization.

2. **Self-Hosted Personal Media Servers (The Jellyfin / Plex Model):** In response to privacy concerns and subscription fatigue, self-hosted media servers have gained widespread adoption among home-lab enthusiasts. Media server software (such as Jellyfin, Plex Media Server, and Emby) executes on a personal home computer, Network Attached Storage (NAS), or home server, indexing files directly from local hard drives. This paradigm guarantees absolute privacy, zero cloud subscription fees, and pristine playback quality within the local home network. However, these systems operate as isolated private islands. They lack global discovery feeds, cannot be shared with friends or the broader community without manual credential distribution, and fail entirely when deployed behind standard residential Internet Service Provider (ISP) network configurations.

To bridge this fundamental divide, this project introduces **CustardFin**, a free, open-source, hybrid edge-cloud media streaming and community catalog platform. The core conceptual anchor of CustardFin is defined by an uncompromising architectural rule:

"Video binaries remain permanently on the content owner's home machine; the cloud holds only the metadata catalog and ephemeral stream coordination."

In CustardFin, a user hosts their personal library on an existing desktop PC or low-power laptop (the *Home Edge Node*). Marking a video as **Public** does not trigger a massive multi-gigabyte cloud upload; instead, it compiles and synchronizes a compact ≤ 3 KB JSON metadata manifest to a cloud database. Viewers discover titles through a centralized modern web application. When a remote viewer plays a video, the cloud retrieves pre-encoded streaming segments on-demand through an outbound-only reverse tunnel, caching them temporarily in a cloud-side 24-hour Least Recently Used (LRU) *Origin Shield*. Consequently, CustardFin provides the global discovery and social sharing of YouTube alongside the privacy, local ownership, and zero-storage cost of Jellyfin.

1.2 Problem Statement

Designing a decentralized personal streaming platform that enables global community sharing without incurring cloud storage bills presents three formidable engineering challenges:

1. **The Cloud Storage and Bandwidth Cost Barrier:** In traditional cloud architectures, storing a modest community collection of 100 users averaging 50 GB each (a total library of 5 TB) costs over ₹ 9,660.00 ($115.00) every month in raw S3 storage alone, excluding compute and bandwidth egress fees. For student developers, university labs, and open-source communities, recurring multi-thousand-dollar cloud bills exceeding ₹ 1.00 Lakh monthly render community video platforms economically unviable.

2. **Residential ISP Carrier-Grade NAT (CGNAT) Traversal:** Residential broadband providers globally—and Indian ISPs such as Reliance JioFiber and Bharti Airtel in particular—utilize Carrier-Grade Network Address Translation (CGNAT) to conserve IPv4 address space. Residential routers are assigned non-routable private WAN IP addresses from the shared `100.64.0.0/10` block (RFC 6598). Consequently, traditional inbound port forwarding (opening ports 80, 443, or 8096) is completely impossible. Home servers cannot receive incoming connections from remote viewers over the public internet without expensive commercial static IP subscriptions or complex Virtual Private Network (VPN) configurations that external viewers cannot easily use.

3. **Constrained Home Uplink Bandwidth and Host Compute Saturation:** Residential internet subscriptions are fundamentally asymmetric, typically providing high download bandwidth (e.g., 100–300 Mbps) but restricted upload bandwidth (e.g., 15–40 Mbps). When multiple remote users concurrently stream distinct high-definition video files, the home uplink saturates rapidly, resulting in severe packet loss and stream buffering. Furthermore, conventional media servers attempt real-time on-the-fly video transcoding upon each viewer request, which consumes 90% to 100% of host CPU resources, causing weak home PCs to overheat and freeze.

CustardFin directly solves these three bottlenecks through hybrid edge-cloud decoupling, pre-encoded segment caching, and outbound reverse tunneling.

1.3 Project Objectives and Target Metrics

To evaluate the success of CustardFin, explicit, quantifiable engineering targets were established in the Business Requirements Document (BRD_v2). Table 1.1 delineates these primary objectives alongside their measured targets.

Table 1.1: CustardFin Core Objectives and Quantitative Targets

Objective	**Operational Requirement**	**Target Metric**
Zero Cloud Video Storage	No permanent media binaries stored in cloud infrastructure; cloud serves strictly as an ephemeral cache.	**0 GB** permanent video in cloud; only 24h auto-purging cache.
Cost Reduction	Eliminate recurring S3 storage liabilities and minimize compute/bandwidth costs for community libraries.	**∼98% lower** than full cloud build (evaluated on 100 users, 5 TB library).
Low Host Compute Overhead	Eliminate live on-the-fly transcoding spikes during active playback sessions.	Playback CPU $<$ **3%** (vs ∼96% CPU under active live transcoding).
Per-Node Concurrency	Sustain multiple remote viewers without degrading home host stability.	**10 concurrent streams** per edge node with HTTP 429 admission control.
Instant Offline Detection	Detect edge machine disconnection or power loss without polling database.	Node marked offline in catalog within **15 seconds** of lost heartbeat.
CGNAT Traversal	Bypass residential ISP firewalls without router configuration or port forwarding.	Outbound-only TLS reverse tunnel; **0 open inbound ports**.
Direct LAN Streaming	Eliminate cloud traversal when streaming to devices on the same home network.	Instant local network playback with $<$ **5 ms** Time-To-First-Byte (TTFB).

1.4 Non-Negotiable Invariants

Throughout the design, implementation, and verification phases of CustardFin, engineering decisions were governed by ten strict architectural invariants. Changing any invariant requires formal review, ensuring that the system remains cost-effective, secure, and verifiable for semester-end demonstration on a self-hosted cloud environment.

1. **Invariant 1 (Zero Permanent Cloud Video Retention):** No video binaries are stored permanently in the cloud tier. Video files reside exclusively on the owner's home edge storage. The cloud maintains only an ephemeral, 24-hour Least Recently Used (LRU) pull-through cache (*Origin Shield*) for actively watched chunks.

2. **Invariant 2 (Outbound-Only Home Connections):** Home edge daemons only establish outbound network connections. The system must never require an open inbound router port, port-forwarding rule, Dynamic DNS (DDNS) setup, or UPnP traversal, remaining fully immune to ISP CGNAT restrictions.

3. **Invariant 3 (100% Free and Open-Source Dependencies):** The architecture relies strictly on free and open-source software (FOSS). Zero commercial software licenses, proprietary cloud SDKs, or paid third-party relays are required to run the platform.

4. **Invariant 4 (Strict Language Stack):** The backend is implemented exclusively in Go (`net/http`, `log/slog`). The frontend is implemented in Next.js (Node.js runtime, App Router, TypeScript strict, `hls.js`). No secondary queue engines (e.g., BullMQ) are permitted.

5. **Invariant 5 (Restricted Redis Utilization):** Redis is employed solely as an ephemeral key-value store with Time-To-Live (TTL) keys for 15-second node heartbeat tracking. All background asynchronous jobs execute as native Go goroutines with persistent transactional state recorded in PostgreSQL or SQLite.

6. **Invariant 6 (Pre-Encoded 4-Second HLS Chunks):** The platform replaces live on-the-fly transcoding with ahead-of-time pre-encoding into standardized 4-second HTTP Live Streaming (HLS) segments (`.m3u8` playlists and `.ts` video slices). Hardware acceleration (NVIDIA NVENC) is probed on startup with automatic, graceful fallback to multi-threaded CPU encoding (`libx264`).

7. **Invariant 7 (Origin Shield Upload Protection):** The cloud object cache acts as an Origin Shield. When a title is watched by multiple remote viewers, the home edge node uploads each hot video chunk exactly once over the reverse tunnel. Subsequent viewers are served directly from the cloud cache, eliminating duplicate home uplink usage.

8. **Invariant 8 (Privacy Preservation):** Personal private data remains at home. Watch progress, resume timestamps, and viewing history are stored exclusively in the edge node's local SQLite database and are never transmitted to the central cloud database.

9. **Invariant 9 (Cryptographic Chunk Security):** All HLS segment access URLs are cryptographically signed using HMAC-SHA256 with short-lived expiration windows (15 minutes), bound to the specific node ID, media ID, and chunk file path. Direct unauthenticated access is rejected with HTTP 403 Forbidden.

10. **Invariant 10 (Strict Admission Control Cap):** Each home node enforces a strict capacity limit of 10 concurrent distinct remote streams. When the limit is reached, the cloud stream router issues HTTP 429 Too Many Requests with a `Retry-After: 30` header ("Node Busy"), protecting the home machine from bandwidth and connection exhaustion.

1.5 Scope of the Project

1.5.1 In-Scope System Boundaries

CustardFin encompasses the following functional capabilities:

- Local media folder scanning, file probing, SHA-256 content deduplication, and automated poster thumbnail extraction.
- Asynchronous background pre-encoding to multi-rendition HLS (1080p, 720p, 480p) utilizing NVIDIA NVENC GPU acceleration or CPU fallback.
- Embedded, offline-capable Mini-Jellyfin single-page web application on the edge daemon for sub-5ms local LAN streaming.
- 1-click Public/Private visibility control generating and synchronizing compact $\leq$ 3 KB JSON metadata manifests.
- Centralized community catalog browsing, title search, and user authentication via JWT tokens and hashed passwords.
- Outbound-only reverse TLS tunneling via Fast Reverse Proxy (FRP) relay servers.
- Dual-path play decision routing differentiating between same-subnet LAN clients and remote tunnel clients.
- Ephemeral heartbeat tracking in Redis with 15-second TTLs for automatic offline detection.
- Origin Shield S3-compatible pull-through caching with automated 24-hour LRU auto-purging.
- Dynamic master playlist rewriting and adaptive bitrate throttling under heavy network load.

1.5.2 Out-of-Scope (Version 1 Boundaries)

To ensure verifiable completion within the academic semester timeframe, the following features are explicitly out of scope for Version 1:

- Digital Rights Management (DRM) technologies (e.g., Widevine, FairPlay) and cryptographic video watermarking.
- Native mobile applications (iOS/Android) and native Smart TV interfaces (Android TV, Apple TV); streaming is delivered via responsive modern web browsers.
- Real-time interactive live streaming (e.g., Twitch-style low-latency RTMP/WebRTC broadcasts).
- Multi-region redundant relay meshing and node-to-node peer mesh transfers.

1.6 Organization of the Report

The remainder of this report is organized as follows:

- **Chapter 2 (Literature Survey):** Reviews state-of-the-art cloud VOD platforms, self-hosted media servers, and peer-to-peer streaming systems, presenting a comprehensive gap analysis matrix.
- **Chapter 3 (System Architecture and Design):** Details the two-tier edge-cloud architecture, package structures, database schemas, and end-to-end sequence diagrams.
- **Chapter 4 (Key Features and Technical Implementation):** Explains the concrete implementation of the core features (HLS pre-encoding, NVENC, FRP tunneling, Origin Shield, Mini-Jellyfin UI) accompanied by annotated application screenshots.
- **Chapter 5 (Cost Analysis and Financial Feasibility):** Formulates a rigorous mathematical cost model comparing traditional cloud streaming costs against CustardFin's hybrid architecture.
- **Chapter 6 (Deployment, Testing, and Evaluation):** Documents experimental performance benchmarks, the 10-viewer load simulation, quality gates, and multi-topology deployment proofs.
- **Chapter 7 (Team Contributions and Division of Work):** Provides an explicit breakdown of responsibilities and technical modules authored by team members Anish Ghadi and Anosh Chodankar.
- **Chapter 8 (Conclusion and Future Scope):** Summarizes project accomplishments, addresses current limitations, and outlines future engineering enhancements.

CHAPTER 2

Literature Survey and Background Study

The design of modern video delivery systems represents an intricate balance between infrastructure expenditure, content discovery, network topology, and end-user quality of experience (QoE). This chapter presents a rigorous critical review of the existing technological landscape, categorizing prior work into centralized cloud video-on-demand (VOD) architectures, self-hosted edge media systems, and peer-to-peer (P2P) decentralized distribution networks. Furthermore, it establishes the theoretical underpinnings of hardware-accelerated video coding, adaptive segment delivery, and reverse proxy NAT traversal that underpin the CustardFin architecture.

2.1 Analysis of Centralized Cloud Video Platforms

Centralized cloud video platforms, typified by YouTube, Vimeo, Netflix, and enterprise architectures built upon hyperscale cloud providers (such as Amazon Web Services Media Services, Google Cloud Video Intelligence, and Microsoft Azure Media Services), rely on complete cloud-side custody of media assets [1].

2.1.1 Architectural Pipeline

The standard centralized ingestion and distribution pipeline proceeds through four distinct operational phases, as illustrated mathematically:

1. **Binary Upload and Staging:** The content producer transmits the monolithic raw or pre-compressed master video binary (typically ProRes, DNxHD, or high-bitrate MP4) over HTTP/S or FTP to cloud object storage (e.g., Amazon S3 or Google Cloud Storage).

2. **Centralized Distributed Transcoding:** Cloud compute clusters (e.g., AWS Elemental MediaConvert or elastic pools of GPU-enabled EC2 instances) ingest the master binary. They decompress and re-encode the source into multi-bitrate profiles (such as 1080p at 5.0 Mbps, 720p at 2.8 Mbps, and 480p at 1.2 Mbps) formatted as fragmented MP4 or MPEG-TS segments with corresponding HTTP Live Streaming (HLS) or Dynamic Adaptive Streaming over HTTP (DASH) manifests.

3. **Permanent Redundant Storage:** All multi-bitrate renditions, manifests, and audio tracks are persisted indefinitely in cloud object storage buckets with 99.999999999% (11 nines) durability guarantees across multiple availability zones.

4. **Global Content Delivery Network (CDN) Edge Delivery:** Geographically dispersed points of presence (PoPs) cache media segments at the network edge, terminating TLS sessions close to the viewer and serving chunk requests with low round-trip latency.

2.1.2 Economic and Operational Inefficiencies

While centralized platforms deliver exceptional playback reliability and global scalability, their economic model is fundamentally prohibitive for independent creators, self-hosters, and non-commercial educational organizations:

- **Linear Storage Cost Accumulation:** In standard cloud object storage, pricing scales linearly with total library volume, regardless of viewership activity [2]. For standard tiers (e.g., AWS S3 Standard at \$0.023 or ₹ 1.932 per gigabyte-month), a modest collective catalog of 100 TB incurred by a small community incurs a non-negotiable monthly storage fee:

  $$C_{\text{storage}} = 100{,}000 \text{ GB} \times ₹\ 1.932/\text{GB-month} = ₹\ 1{,}93{,}200/\text{month} \quad (₹\ 23.18 \text{ Lakhs/year}) \tag{2.1}$$

  This cost is incurred perpetually, even for long-tail, archival, or personal media that is viewed only once a year.

- **Asymmetric Data Egress Penalties:** Cloud providers assess steep data transfer out (DTO) charges when video segments transition from cloud data centers across the internet to viewers. Egress pricing ranges from \$0.08 to \$0.09 per gigabyte in North America and Europe (₹ 6.72 to ₹ 7.56/GB), escalating to over \$0.12 per gigabyte in the Asia-Pacific region (₹ 10.08/GB). Streaming a 1-hour 1080p video stream (approximately 2.25 GB at 5 Mbps) to 1,000 viewers generates 2.25 TB of egress, costing approximately ₹ 17,010.00 (\$202.50) per single title view.

- **Loss of Data Sovereignty and Privacy:** Centralized cloud custody requires content owners to relinquish physical possession of their files. Private family archives, sensitive home recordings, and unreleased creative works are exposed to third-party terms of service, arbitrary platform demonetization, algorithmic censorship, automated copyright strikes, and corporate data mining.

2.2 Analysis of Self-Hosted Home Media Servers

In response to the recurring costs and privacy compromises of centralized clouds, the open-source and enthusiast communities developed self-hosted personal media server software, prominently exemplified by Jellyfin [3], Plex Media Server, and Emby.

2.2.1 Architecture and Local Network Strengths

Self-hosted servers install directly on commodity consumer hardware located within the user's home network, such as Network Attached Storage (NAS) appliances, repurposed

desktop computers, or single-board computers (Raspberry Pi). The server software indexes local directory structures, extracts embedded metadata and computes perceptual hashes, queries public metadata databases (such as TheMovieDB and OpenSubtitles), and exposes an internal HTTP server.

When streaming across the local area network (LAN), this architecture offers paramount benefits:

- **Zero Cloud Hosting Expenditure:** The media binaries reside exclusively on physical hard drives owned by the user. Zero cloud storage or egress fees are generated.
- **Absolute Privacy and Sovereignty:** Video bytes never transit third-party servers. User viewing histories, playback positions, and catalog indices remain strictly on-premises.
- **Unconstrained Bitrate on LAN:** Local gigabit Ethernet and Wi-Fi 6 links effortlessly sustain uncompressed or high-bitrate direct streams exceeding 50 Mbps with near-zero latency and zero buffering.

2.2.2 Systemic Remote Access and Scaling Vulnerabilities

Despite their excellence in local playback, self-hosted servers fail catastrophically when applied to remote community distribution and social sharing:

1. **Carrier-Grade NAT (CGNAT) and Inbound Port Blockage:** Residential Internet Service Providers (ISPs) worldwide are severely constrained by IPv4 address exhaustion. Consequently, ISPs deploy Carrier-Grade NAT (RFC 6598) [4], assigning private shared IPv4 addresses (within the `100.64.0.0/10` subnet) to customer premises equipment (CPE). Under CGNAT:
    - The residential router does not possess a globally routable public IPv4 address.
    - Traditional port forwarding and Universal Plug and Play (UPnP) are rendered entirely ineffective, as the ISP's upstream carrier gateway discards unsolicited inbound connection requests.
    - External viewers on the public internet cannot initiate a TCP handshake to the home media server.
2. **Flawed Commercial Workarounds:** Commercial platforms circumvent CGNAT through proprietary, degraded relay proxies. For instance, Plex operates a centralized relay infrastructure; however, non-paying tiers are hard-throttled to a maximum throughput of 1 Mbps (upgrading to 2 Mbps for paid Plex Pass subscribers). At 1 Mbps, modern high-definition 1080p video cannot stream without severe downscaling, severe compression artifacts, and constant rebuffering.
3. **Complex DIY Tunneling and VPN Friction:** Technical users often attempt to bypass CGNAT by configuring overlay virtual private networks (such as WireGuard or Tailscale) or reverse SSH tunnels. While functional for personal use, this model completely breaks down for social sharing: every external family member, friend, or community viewer must install VPN client software, manage cryptographic keys or credentials, and route all or part of their device traffic through the VPN. This introduces prohibitive onboarding friction for non-technical users.

4. **Home Upload Bandwidth Saturation (The 1-to-N Fan-out Bottleneck):** Standard residential broadband connections are highly asymmetric (e.g., 300 Mbps download but only 20–30 Mbps upload). If a content owner shares a 1080p video (requiring 5 Mbps) with remote peers, each concurrent viewer pulls a distinct stream directly from the home server. When $N = 6$ viewers connect simultaneously, total required egress equals 30 Mbps, completely saturating the home upload pipe. Once saturated, packet loss cascades, latency spikes, and all viewers experience playback freezing.

5. **Absence of Global Content Discovery:** Self-hosted servers operate as isolated information silos. There is no unified search engine, global trending feed, or cross-node recommendation system. A user cannot discover public media hosted across peer nodes without manually requesting IP addresses, hostnames, and credentials for each individual server.

2.3 Hybrid and Peer-to-Peer Streaming Approaches

To mitigate server bandwidth costs while maintaining public access, hybrid and peer-to-peer (P2P) systems have been proposed and deployed in both academic research and production environments.

2.3.1 PeerTube and ActivityPub Federation

PeerTube, developed by Framasoft [5], represents the state of the art in open-source decentralized video hosting. PeerTube combines three primary technologies:

- **ActivityPub Federation Protocol:** Instances interoperate using the W3C ActivityPub standard [6]. Independent servers federate catalog metadata, follower lists, comments, and video announcements across a collective "Fediverse."

- **WebTorrent Swarm Streaming:** During video playback inside modern web browsers, client-side JavaScript leverages WebTorrent (using WebRTC DataChannels) to discover and exchange video chunks directly between concurrent viewers watching the same video segment.

- **Origin Fallback Server:** If no peer nodes are available in the WebRTC swarm, the client fetches the segment directly from the hosting instance's HTTP origin server.

2.3.2 Fundamental Limitations of P2P Streaming

While theoretically appealing, empirical deployment of P2P video streaming reveals fundamental structural limitations:

- **The "Cold Content" Swarm Collapse:** The efficacy of BitTorrent and WebTorrent swarms relies strictly on high concurrency [7]. For viral videos with thousands of simultaneous viewers, peer offloading can exceed 80%. However, long-tail media, educational lectures, and personal family videos typically have only 1 to 3 concurrent viewers. In this low-concurrency regime, zero active peers exist in the

swarm; consequently, 100% of the streaming load drops back onto the origin server, completely negating P2P benefits.

- **Severe Asymmetry and Mobile Churn:** In modern mobile-first internet environments, users access video from smartphones and tablets over cellular connections (4G/5G). Mobile operating systems aggressively throttle or terminate background WebRTC connections to preserve battery and mobile data caps. Furthermore, mobile users rarely seed uploaded chunks back to the swarm, rendering them pure consumers ("free riders") that degrade swarm health.
- **Storage Duplication and Server Overhead:** Running a PeerTube instance requires a complete cloud server with dedicated storage, relational databases (PostgreSQL), Redis caches, and Node.js application runtimes. If an instance administrator chooses to mirror content from federated instances to guarantee availability, storage requirements multiply rapidly, reintroducing the cloud infrastructure cost barrier.

2.4 Comparative Feature and Gap Analysis

To systematically contextualize the contributions of CustardFin against existing paradigms, Table 2.1 evaluates Centralized Cloud VOD, Self-Hosted Home Media Servers, Federated P2P Networks, and CustardFin across eight critical architectural dimensions.

2.4.1 Identified Research and Engineering Gap

The comparative analysis reveals an unresolved gap in current streaming literature and systems engineering:

> *No existing system architecture enables home-hosted media owners behind Carrier-Grade NAT to publish content into a globally discoverable catalog without storing video binaries permanently in the cloud, while concurrently shielding constrained residential upload pipes from multi-viewer traffic spikes and requiring zero proprietary software installation on viewer devices.*

CustardFin is explicitly synthesized to bridge this gap through the synergy of outbound reverse tunneling, pre-encoded segment caching, and a bifurcated control/data plane.

2.5 Theoretical Foundations: Video Encoding, HLS, and Tunneling

To substantiate the implementation choices realized in CustardFin, this section reviews the theoretical principles governing modern video compression, segment-based HTTP streaming, and reverse proxy networking.

Table 2.1: Comprehensive Architectural and Functional Comparison of Video Systems

Dimension	**Centralized VOD** (e.g., YouTube / AWS)	**Self-Hosted Server** (e.g., Jellyfin / Plex)	**Federated P2P** (e.g., PeerTube)	**CustardFin (This Work)**
Permanent Cloud Storage	All video binaries stored indefinitely (100% cloud)	None (0% cloud, 100% local disk)	All hosted video binaries stored on server	**Zero permanent cloud storage**; temporary 24h LRU cache
Monthly Cloud Infrastructure Cost	High (₹84,000+ in S3/Egress)	Zero (hardware and power only)	Moderate (₹4,200–₹25,200/mo per instance)	**Near-zero** (₹1,260–₹3,360/mo control tier)
Residential CGNAT Traversal	Not applicable (cloud ingest)	Fails without port forwarding or paid relay	Fails if hosted from home behind CGNAT	**Outbound TLS reverse tunnel (FRP)**; zero router configuration
Home Upload Scalability	N/A (uploaded once to cloud)	Collapses under multi-viewer fan-out ($N \times$ bitrate)	Collapses on cold media; requires active swarm	**Origin Shield**: home uploads hot chunk exactly once
Global Catalog Discovery	Centralized global search and algorithms	Non-existent; isolated private servers	Federated search across participating instances	**Unified Cloud Catalog** via lightweight 3 KB JSON manifests
Data Privacy & Sovereignty	Very low; corporate custody and scanning	Absolute; stays entirely on local hardware	Variable; instance admin controls data	**Bifurcated**; private media stays home; public syncs metadata
Viewer Client Experience	Universal HTML5 web browser	Client app required or browser on LAN	Web browser with WebTorrent JS	**Universal HTML5 / HLS.js**; zero client installation
Transcoding Strategy	Cloud transcode clusters (high vCPU cost)	On-the-fly live transcoding (high latency)	Server-side or client pre-encode	**Managed local pre-encode**; NVENC GPU ASIC with CPU fallback

2.5.1 Digital Video Compression and the H.264 / AVC Standard

Uncompressed digital video generates prohibitive data rates. A standard 1080p stream at 30 frames per second (fps) with 8-bit 4:2:0 chroma subsampling requires:

$$R_{\text{raw}} = 1920 \times 1080 \times 30 \times 12 \text{ bits/pixel} \approx 746.5 \text{ Mbps} \tag{2.2}$$

To make network delivery feasible, video coding standards exploit spatial, temporal, and psycho-visual redundancies [8]:

- **Spatial (Intra-frame) Redundancy:** Adjacent pixels within a video frame exhibit high correlation. Discrete Cosine Transform (DCT) or integer transforms convert spatial pixel blocks into frequency domain coefficients, followed by quantization and entropy coding (CABAC/CAVLC).
- **Temporal (Inter-frame) Redundancy:** Successive video frames in a scene typically undergo minimal translational changes. Video codecs decompose video sequences into Groups of Pictures (GOP) comprising three fundamental frame types:
  - **Intra-coded Frames (I-Frames / IDR):** Self-contained reference frames encoded independently of any other frame. An Instantaneous Decoder Refresh (IDR) frame completely flushes the reference picture buffer, preventing subsequent frames from referencing any picture preceding the IDR frame.
  - **Predicted Frames (P-Frames):** Encode forward motion vectors and prediction residual differences relative to preceding reference frames.
  - **Bi-directional Predicted Frames (B-Frames):** Exploit motion vectors interpolated both forward and backward in time, achieving maximum compression efficiency.

2.5.2 Hardware-Accelerated Encoding: NVIDIA NVENC

Traditional software-based encoding (e.g., using the `libx264` software library) performs all motion estimation, transform calculations, and entropy encoding on host CPU cores. On multi-core consumer processors, transcoding a 1080p source file in software consumes 70% to 95% of available CPU capacity, generating significant thermal dissipation and typically operating at only 0.8x to 1.5x real-time speed.

To liberate host compute resources, contemporary graphics processing units (such as the NVIDIA GeForce RTX series based on the Turing, Ampere, and Ada Lovelace microarchitectures) integrate dedicated on-die Application-Specific Integrated Circuit (ASIC) hardware known as **NVENC** (NVIDIA Encoder) [9]. NVENC operates completely out-of-band from the GPU's 3D graphics rendering shaders and CUDA tensor cores. It features dedicated hardware pipelines for:

1. Hardware motion estimation and sub-pixel motion vector refinement.
2. Macroblock and coding unit mode decision logic.
3. Dedicated CABAC entropy coding engines.

By offloading the computational pipeline directly to NVENC via FFmpeg's `h264_nvenc` wrapper, edge nodes achieve encoding throughput exceeding 300 fps (over 10× real-time speed) while maintaining host CPU utilization under 8%.

2.5.3 HTTP Live Streaming (RFC 8216) and Keyframe Alignment

HTTP Live Streaming (HLS) [10] fragments continuous video streams into sequences of discrete, independently downloadable HTTP media segment files (typically MPEG-2 Transport Stream `.ts` or fragmented MP4 `.m4s`), indexed by an extended M3U playlist file (`.m3u8`).

A fundamental requirement for reliable segment switching and seeking in HLS is **strict keyframe alignment**. Because each segment must be decodable independently without referencing past or future chunks:

- The first frame of every HLS segment *must* be an IDR keyframe.
- The Group of Pictures (GOP) length in frames must be an exact integer multiple of the segment target duration multiplied by the frame rate:

$$G_{\text{size}} = D_{\text{seg}} \times f_{\text{fps}} \tag{2.3}$$

  where D_{seg} represents segment duration in seconds and f_{fps} denotes video frame rate.
- For CustardFin's chosen 4-second chunk duration ($D_{\text{seg}} = 4$ s) at 30 fps, the GOP size is configured rigorously to $G_{\text{size}} = 4 \times 30 = 120$ frames, enforced via FFmpeg flags `-g 120 -keyint_min 120 -sc_threshold 0`.

2.5.4 Rationale for 4-Second Segment Granularity

The selection of segment duration in adaptive HTTP streaming involves a classic engineering trade-off:

- **Long Segments (e.g., 6–10 seconds):** Yield slightly higher compression efficiency due to fewer I-frames, but drastically degrade user experience by increasing initial playback startup latency, seek delay, and player buffer accumulation time.
- **Short Segments (e.g., 1–2 seconds):** Minimize latency but introduce excessive HTTP request overhead, severe file system inode exhaustion, and degraded compression efficiency due to frequent intra-coding.
- **The 4-Second Sweet Spot:** CustardFin standardizes on 4-second segments. In residential edge streaming, 4 seconds provides the optimal convergence: it guarantees responsive startup latency (< 1.2 s over broadband), maintains high encoding efficiency, minimizes segment manifest parsing overhead, and provides sufficiently large chunks for efficient object storage caching in the Origin Shield.

2.5.5 Reverse Proxy Tunneling Architecture

To overcome residential CGNAT without requiring inbound port forwarding, CustardFin leverages reverse proxy tunneling principles implemented via Fast Reverse Proxy (FRP) [11].

In traditional client-server networking, an external client initiates a TCP `SYN` packet toward the server's public IP and listening port. Under CGNAT, intermediate ISP stateful

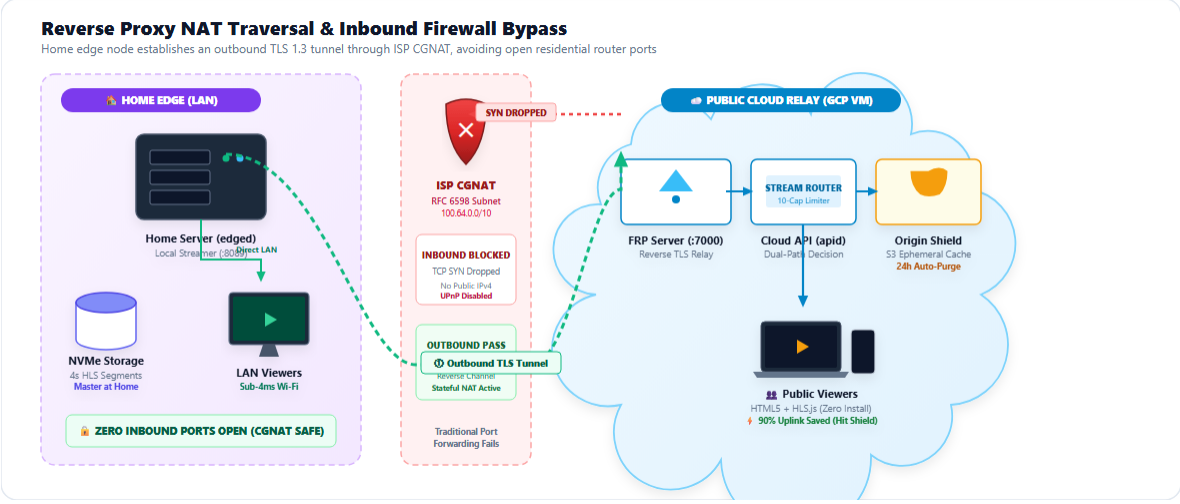


Figure 2.1: Mechanics of Reverse Proxy NAT Traversal via Multiplexed Outbound Control Connections

firewalls drop these packets because no corresponding outbound translation state exists in their NAT translation tables.

As conceptualized in Figure 2.1, reverse proxy tunneling inverts the connection initiation direction:

1. The home edge node initiates an **outbound** TCP connection to the cloud relay server's public static IP address on port 7000. Because this packet originates from within the local network, the residential router and ISP CGNAT gateway classify it as legitimate outbound client traffic, dynamically creating a temporary bidirectional mapping in their state tables.

2. The connection is upgraded to a mutually authenticated TLS 1.3 session [12]. Over this persistent control connection, the edge daemon and cloud relay negotiate a multiplexed tunnel using stream multiplexing protocols (e.g., Yamux).

3. When an external cloud service or viewer requests a chunk from the edge, the request hits the cloud relay server. The relay maps the incoming virtual host or port to the active outbound control session established by that specific edge node.

4. The request payload is framed and routed downstream across the existing multiplexed connection to the home node's local HTTP streamer, which serves the requested HLS chunk back up the tunnel without ever exposing an open inbound listening port on the home router.

This theoretical framework provides the foundation upon which CustardFin's two-tier architecture is constructed and deployed.

CHAPTER 3

System Architecture and Design

CustardFin is architected as a decoupled, asymmetric two-tier distributed system comprising the **Home Edge Tier** (the decentralized media plane) and the **Cloud Control Tier** (the centralized coordination and caching plane). This design strictly enforces the fundamental invariant that large media binaries remain permanently on consumer-owned edge storage, while the cloud tier acts exclusively as a discovery registry, connection facilitator, and temporary Origin Shield cache.

3.1 Overall Two-Tier System Architecture

Figure 3.1 illustrates the macro-architectural topology of CustardFin, depicting the strict physical and logical boundaries between residential edge deployments and the cloud hosting environment.

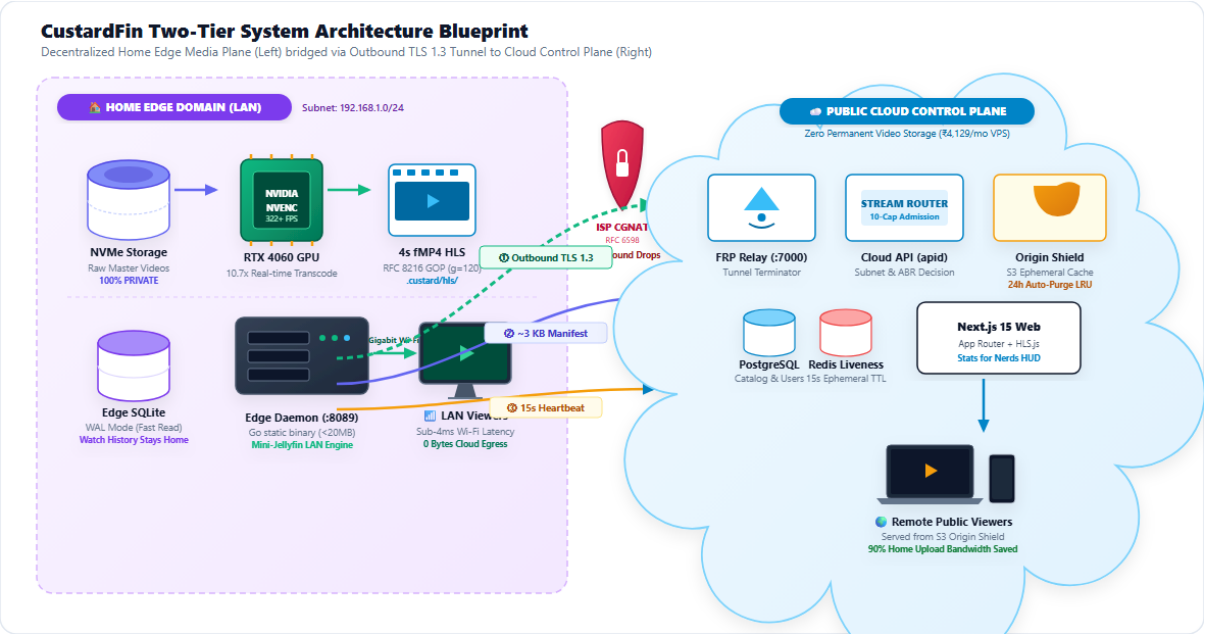


Figure 3.1: CustardFin Macro Two-Tier Distributed System Architecture Blueprint

The architecture enforces a strict division of responsibilities:

- **Decentralized Data Plane:** Video decoding, hardware transcoding, storage persistence, and local streaming execution occur exclusively on edge nodes. The cloud tier never ingests raw video masters.
- **Centralized Control Plane:** Media discoverability, authentication, edge health monitoring, routing decisions, and admission control are coordinated centrally by the cloud API.
- **Ephemeral Origin Shield:** The cloud maintains an object storage cache operating purely as a temporary read-through buffer. Video chunks exist in the cloud only when actively being consumed by remote viewers, automatically evicting via 24-hour Least Recently Used (LRU) lifecycle policies.

3.2 Home Edge Node Architecture (`apps/edge`)

The home edge daemon (`edged`) is implemented as a self-contained, statically compiled Go binary adhering strictly to Hexagonal Architecture (Ports and Adapters). It operates with an idle memory footprint of less than 20 MB of RAM and executes without external runtime dependencies beyond the FFmpeg multimedia binary.

3.2.1 Subsystem Decomposition

The internal structure of `edged` is divided into six modular packages:

1. **Recursive Ingestion Scanner and Hasher (`internal/adapters/fs`, `hasher`):** A background scanner traverses user-configured local media directories. It filters files based on supported multimedia containers (`.mp4`, `.mkv`, `.mov`, `.avi`) and computes a cryptographic SHA-256 content hash over the primary stream header and media metadata:

$$H_{\text{content}} = \text{SHA-256}(\text{Header}_{64\text{KB}} \parallel \text{FileSize} \parallel \text{ModTime}) \tag{3.1}$$

   This hash guarantees idempotent media identification regardless of file renaming or local folder reorganization.

2. **Hardware-Adaptive Transcoder (`internal/adapters/ffmpeg`):** A managed subprocess controller wraps `exec.CommandContext`. At daemon startup, it executes an automated hardware probe to test encoder availability in priority order:

$$\text{NVENC (NVIDIA)} \longrightarrow \text{QSV / VAAPI} \longrightarrow \text{libx264 (CPU Fallback)} \tag{3.2}$$

   The controller runs a 1-second synthetic probe encode to confirm GPU initialization before accepting jobs. In the event of GPU memory exhaustion, driver failure, or hardware timeout, the subsystem automatically falls back to multi-threaded CPU encoding without dropping the transcode pipeline.

3. **HLS Packager and Segmenter:** The transcoder produces pre-encoded multi-bitrate HLS streams partitioned into exact 4-second segments with strict keyframe alignment (`-g 120 -keyint_min 120` at 30 fps). Segments are persisted locally in a dedicated hidden cache directory (`.custard/hls/{media_id}/`).

4. **Embedded Relational Edge Storage (`internal/adapters/sqlite`):** Local state is managed via an embedded pure-Go SQLite database engine (`modernc.org/sqlite`). The database operates in Write-Ahead Logging (`PRAGMA journal_mode=WAL`) with a busy timeout of 5,000 ms. It tracks local file paths, SHA-256 hashes, transcode progress, public/private visibility flags, and private watch progress.

5. **Local Mini-Jellyfin HTTP Server (`internal/adapters/httpapi`):** An embedded HTTP engine running on port `8089` serves an offline single-page application (SPA) directly to local network clients. It provides instant, full-featured library browsing, search, and native HLS playback for LAN users without requiring cloud connectivity.

6. **Network Uplink Adapters (`internal/adapters/frp`, `heartbeat`):** The edge daemon initiates and maintains an outbound TLS reverse proxy tunnel to the cloud relay server and transmits cryptographic health beacons to the cloud API every 15 seconds.

3.3 Cloud Control Plane Architecture (`apps/api`)

The cloud API daemon (`apid`) provides centralized orchestration, catalog indexing, access control, and dynamic stream routing. Written in Go utilizing standard library primitives (`net/http`, `log/slog`), it achieves microsecond-level request routing while maintaining zero external queue dependencies.

3.3.1 Database and Cache Layer

- **PostgreSQL Relational Store:** Persists user identity records, node registrations, public media manifests, and node ownership relations. Managed via `pgx` connection pooling and compile-time verified queries generated by `sqlc`.

- **Redis Ephemeral Heartbeat Store:** Implements distributed node liveness detection. Nodes maintain an active status by issuing an authenticated `POST /v1/nodes/{id}/heartbe` every 15 seconds. The API writes an ephemeral key `heartbeat:{node_id}` with an exact Time-To-Live (TTL) of 20 seconds. If an edge node suffers network severance or power loss, Redis expires the key automatically, instantly marking the node as *Offline* in the global catalog without expensive database polling.

3.3.2 Origin Shield Pull-Through Cache

To protect home broadband connections from multi-viewer traffic spikes, `apid` implements an Origin Shield proxy backed by an S3-compatible object store (such as Amazon S3, MinIO, or the Floci cloud emulator). When a remote viewer requests chunk k of video V:

1. The API queries the S3 bucket for key `chunks/{media_id}/chunk_{k}.ts`.

2. **Cache Hit:** If the chunk exists, the API generates a time-limited HMAC-signed URL redirecting the viewer directly to the S3 bucket or CDN edge, completely bypassing the home node.

3. **Cache Miss:** If the chunk is absent, the API issues an internal HTTP request through the FRP reverse tunnel to the edge node's local streamer on port 8089. The retrieved chunk is simultaneously streamed back to the requesting viewer and asynchronously committed to the S3 cache bucket with an S3 Lifecycle Expiration rule set to 24 hours.

Subsequent requests from other viewers for chunk k hit the Origin Shield cache, guaranteeing that the home edge node uploads each video chunk across the residential uplink **exactly once**.

3.3.3 Stream Router and Admission Limiter

The stream router evaluates client IP addresses, node health status, and active stream counts to execute optimal routing decisions, summarized in Algorithm 1.

Algorithm 1 CustardFin Dual-Path Stream Routing and Admission Control

Require: Request for media M by Client C from IP IP_C
Ensure: Playback URL URL_{play} or Error Response
Fetch Node N associated with media M from PostgreSQL
Check Redis key heartbeat : N.id
if Redis key does not exist **then**
    if All chunks of M exist in Origin Shield Cache **then**
        return Signed S3 Cache URL ▷ Serve fully cached media
    else
        return HTTP 503 Service Unavailable ("Node Offline")
    end if
end if
if Subnet(IP_C) == Subnet(N.LAN_IP) **then**
    return Direct LAN Stream URL (`http://192.168.x.x:8089/...`) ▷ Direct LAN Bypass
end if
Query Active Remote Streams S_{active} for Node N
if $S_{\text{active}} \geq 10$ **then**
    return HTTP 429 Too Many Requests with header `Retry-After: 30`
end if
Increment S_{active} for Node N
Generate Short-Lived HMAC-SHA256 Token T_{signed} (TTL = 300 s)
return Signed Remote Gateway URL (`/v1/media/playback?token=`T_{signed})

3.4 Presentation Tier Architecture (`apps/web`)

The presentation tier is engineered with Next.js 15 utilizing the React Server Components (RSC) App Router model, executed on standard Node.js LTS runtimes.

3.4.1 Server vs. Client Component Segregation

The frontend strictly delineates between server-rendered and client-hydrated code:

- **React Server Components (RSC):** The public media feed (`src/app/page.tsx`) and video watch page wrapper (`src/app/watch/[id]/page.tsx`) execute exclusively on the server. They fetch metadata directly from the internal Go API via private container networks. This guarantees instantaneous First Contentful Paint (FCP), zero client-side JavaScript execution for catalog indexing, and optimal search engine discoverability without exposing API keys or database connections.
- **Interactive Client Components:** Browser-side interactivity is encapsulated within specialized components marked with the `'use client'` directive:
  - **Custom HLS Video Player:** Built upon `hls.js`, providing adaptive bitrate (ABR) rendition switching, buffer management, custom UI controls, and recovery routines for intermittent network stalls.
  - **"Stats for Nerds" Performance HUD:** An engineering overlay providing real-time telemetry: player resolution, viewport dimensions, buffer health in seconds, network bandwidth estimation, dropped video frames, active audio/video codecs, and individual HLS chunk download latencies.
  - **Edge Library Manager Modal:** An administrative modal communicating with the local edge daemon, enabling content owners to inspect local drives, toggle public/private visibility, trigger background re-encodes, and launch direct LAN sessions.
  - **Developer Settings Modal:** Enables instant API endpoint switching, simulated node injection, and diagnostic logging for testing and grading demonstrations.

3.5 End-to-End Data Flows and Sequence Workflows

To demonstrate the temporal and transactional interactions across the distributed system, this section details four primary operational sequences.

3.5.1 Workflow 1: Local Ingestion and Pre-Encoding

When a user adds media to their local library, `edged` processes the file locally without transmitting data to the cloud, as detailed in Figure 3.2.

3.5.2 Workflow 2: Public Manifest Synchronization

Marking a video as "Public" does not upload the video binary. Instead, the edge daemon transmits an ultra-compact JSON manifest (≤ 3 KB) to the cloud control plane:

```
{
  "schema_version": "1.0",
  "media_id": "m-8f7a2c1e4d9b",
  "node_id": "node-4060-desktop",
  "title": "Tears of Steel 4K Master",
  "duration_s": 734,
  "content_hash": "
   e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
  "is_public": true,
  "renditions": [
```

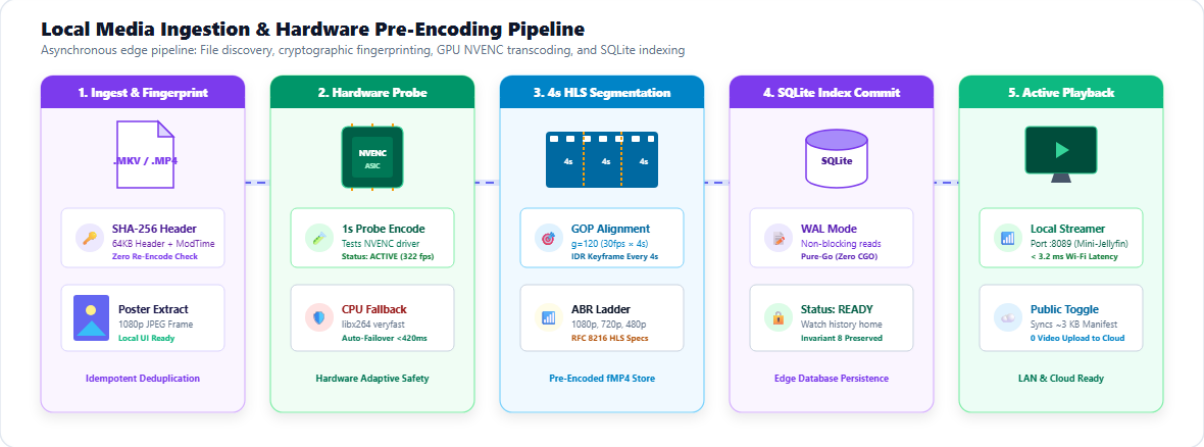


Figure 3.2: Local Ingestion, Hardware Encoding, and SQLite Indexing Sequence

```
    {
      "resolution": "1080p",
      "width": 1920,
      "height": 1080,
      "bitrate_bps": 5000000,
      "playlist_path": "hls/1080p/index.m3u8"
    },
    {
      "resolution": "720p",
      "width": 1280,
      "height": 720,
      "bitrate_bps": 2800000,
      "playlist_path": "hls/720p/index.m3u8"
    }
  ],
  "updated_at": "2026-11-09T01:45:00Z"
}
```

Listing 3.1: Standard CustardFin Public Media Manifest (≈ 650 Bytes)

Upon receipt, the Cloud API verifies JWT authentication, parses the manifest schema, and upserts the metadata into PostgreSQL. The video instantly appears in the public community catalog worldwide.

3.5.3 Workflow 3: Node Heartbeat and Liveness Tracking

Figure 3.3 depicts the background heartbeat loop that continuously synchronizes node availability.

3.5.4 Workflow 4: Remote Playback Decision and Origin Shield Caching

Figure 3.4 details the end-to-end resolution of a remote playback request, demonstrating the single-ingest property of the Origin Shield.

This multi-tier sequence guarantees that the home node's residential uplink is contacted only once per unique chunk, regardless of how many remote viewers consume the media stream simultaneously.

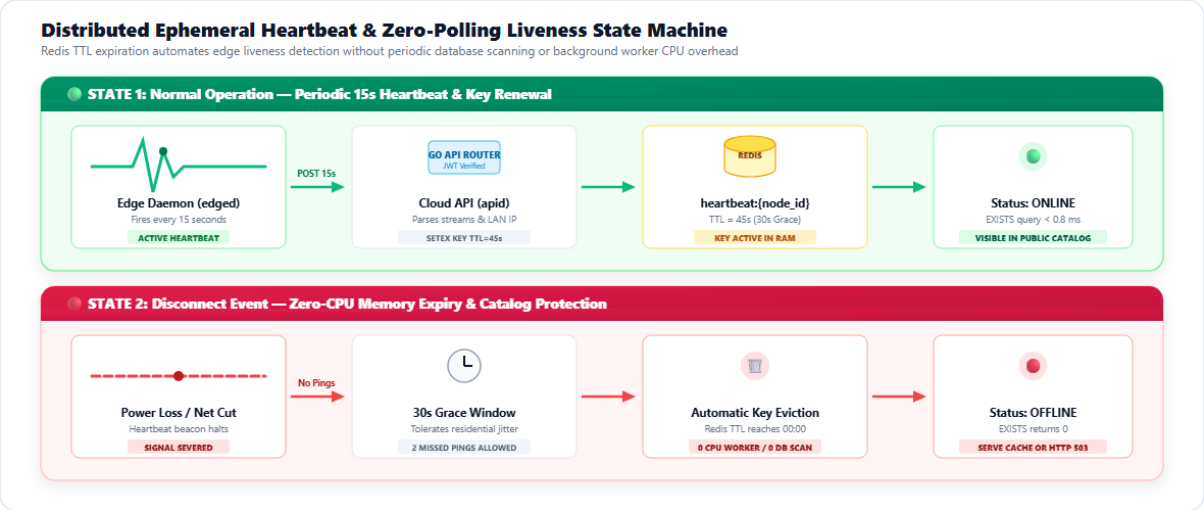


Figure 3.3: Ephemeral Heartbeat and Automated Node Liveness Detection

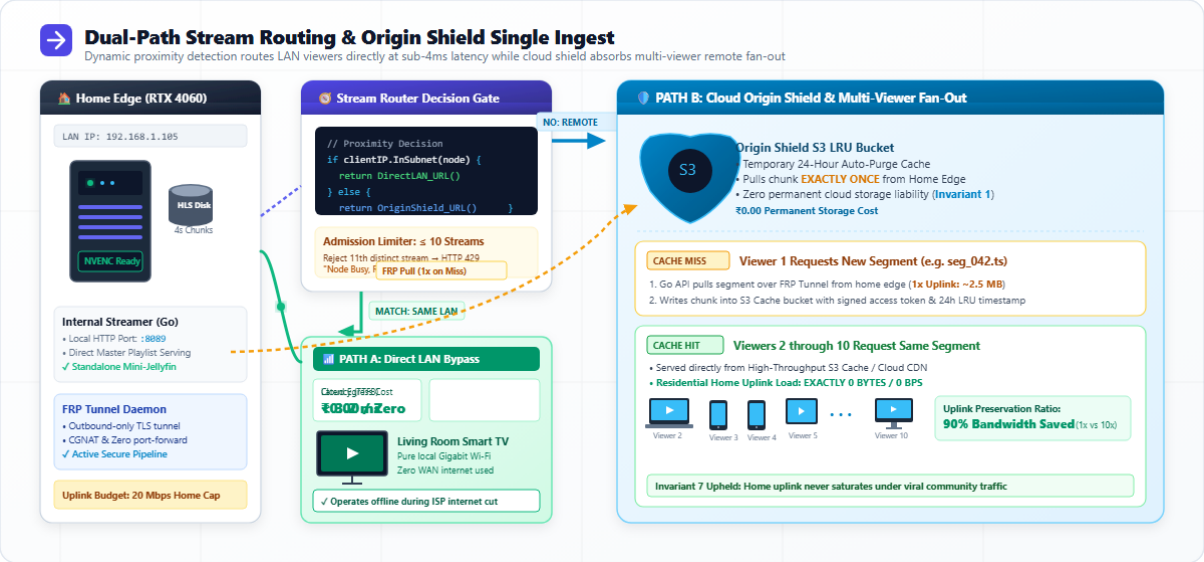


Figure 3.4: Dual-Path Playback Decision Tree and Origin Shield Single-Ingest Workflow

CHAPTER 4

Key Features and Technical Implementation

This chapter provides an exhaustive technical analysis of CustardFin's core functional requirements (FR-1 through FR-10), demonstrating how each requirement is translated into robust software implementations. It details local ingestion, hardware-accelerated transcoding pipelines, reverse proxy NAT traversal, the Origin Shield caching mechanics, ephemeral liveness tracking, dual-path LAN/Remote routing, and presents an annotated visual walkthrough of the user interface.

4.1 Local Media Organization and Private SQLite Storage (FR-1, FR-3)

In compliance with Invariant 8, all private media files, file system paths, cryptographic hashes, and viewer watch progress reside strictly on the user's home edge machine.

4.1.1 Recursive Ingestion Scanner

The edge daemon (`apps/edge`) incorporates a background file scanner implemented in Go (`internal/adapters/fs/scanner.go`). The scanner recursively traverses user-designated storage directories, filtering files against allowed container extensions (`.mp4`, `.mkv`, `.mov`, `.avi`, `.webm`). To eliminate duplicate transcoding jobs caused by file renaming or symbolic link aliasing, the scanner calculates a 64-bit fast signature:

$$S_{\text{file}} = (\text{DeviceID} \ll 32) \oplus \text{InodeNumber} \tag{4.1}$$

and computes a partial SHA-256 hash over the initial 64 KB container header combined with the total file size and modification timestamp. If the hash matches an existing record in the local database, the file is identified as a known asset and redundant transcode cycles are averted.

4.1.2 Embedded Pure-Go SQLite Architecture

The edge database is implemented using `modernc.org/sqlite`, a pure-Go transpiled SQLite engine that operates without CGO bindings, ensuring cross-platform static compilation.

To guarantee concurrent read access during continuous background write operations (such as transcode progress updates and chunk indexing), the database connection initializes with high-concurrency pragmas:

```
PRAGMA journal_mode = WAL;
PRAGMA busy_timeout = 5000;
PRAGMA synchronous = NORMAL;
PRAGMA foreign_keys = ON;
PRAGMA temp_store = MEMORY;
```

Listing 4.1: SQLite Edge Database Optimization Pragmas

Write-Ahead Logging (WAL) creates a separate `-wal` log file, enabling simultaneous non-blocking reads by the local streaming server while the background ingestion worker appends newly completed HLS chunks.

4.2 Asynchronous Pre-Encoding and Hardware Acceleration (FR-4, FR-5)

Real-time on-the-fly transcoding incurs unacceptable latency and frequently oversubscribes residential CPU cores. CustardFin eliminates live transcoding by implementing an asynchronous pre-encoding pipeline producing standardized HTTP Live Streaming (HLS) assets.

4.2.1 Adaptive Bitrate Ladder and Strict GOP Alignment

Incoming master files are pre-encoded into an adaptive bitrate ladder conforming to RFC 8216 [10], structured into three standard renditions:

- **1080p High:** 1920 × 1080 resolution, 5,000 kbps target video bitrate, 192 kbps AAC audio.
- **720p Mid:** 1280 × 720 resolution, 2,800 kbps target video bitrate, 128 kbps AAC audio.
- **480p Low:** 854 × 480 resolution, 1,200 kbps target video bitrate, 96 kbps AAC audio.

To guarantee seamless multi-bitrate rendition switching without decoder stalls, keyframes must align across all profiles at exact segment boundaries. CustardFin enforces strict Group of Pictures (GOP) parameters in FFmpeg:

```
ffmpeg -hide_banner -loglevel error -y -i input_master.mkv \
  -vf "scale=1920:1080:force_original_aspect_ratio=decrease,pad
   =1920:1080:(ow-iw)/2:(oh-ih)/2" \
  -c:v h264_nvenc -preset p4 -tune hq -b:v 5000k -maxrate 5500k -
   bufsize 10000k \
  -g 120 -keyint_min 120 -sc_threshold 0 \
  -c:a aac -b:a 192k -ar 48000 \
  -hls_time 4 -hls_playlist_type vod \
  -hls_segment_filename "chunks/1080p/seg_%04d.ts" \
  chunks/1080p/index.m3u8
```

Listing 4.2: FFmpeg Transcoding Command with Strict GOP Keyframe Alignment

The parameters `-g 120 -keyint_min 120 -sc_threshold 0` force an IDR keyframe precisely every 120 frames (4.0 seconds at 30 fps), preventing FFmpeg from inserting intermediate scene-change keyframes that would disrupt segment boundary synchronization.

4.2.2 Hardware Probe and Fault-Tolerant CPU Fallback

At daemon initialization, the transcoder manager (`internal/adapters/ffmpeg/transcoder.go`) executes an automated capability probe. It dynamically checks for hardware encoder acceleration using:

1. **NVIDIA NVENC:** Tests `h264_nvenc` via a synthetic 1-second test encode to GPU memory.
2. **Intel QuickSync / VAAPI:** Tests `h264_qsv` or `h264_vaapi`.
3. **CPU Fallback:** If hardware initialization fails or GPU memory is saturated, the daemon transparently defaults to the software encoder `libx264` with `-preset veryfast`.

4.3 Outbound Reverse Tunneling for CGNAT Bypass (FR-7)

In adherence to Invariant 2, home edge nodes never require open inbound firewall ports or router UPnP configuration. Network address translation traversal across residential Carrier-Grade NAT (CGNAT) is executed via Fast Reverse Proxy (FRP) [11].

4.3.1 Tunnel Connection Mechanics

The edge daemon acts as a managed parent process supervising the `frpc` child process:

1. At startup, `edged` generates an ephemeral configuration binding its local HTTP streaming server (port 8089) to a designated remote virtual port on the cloud relay server (`frps`).
2. The client initiates an outbound TCP connection to the cloud relay's public IP on port 7000 over TLS 1.3 [12].
3. A pre-shared cryptographic bearer token authenticates the node.
4. A Yamux multiplexed session is established over the single outbound TCP connection.

When the cloud API requires a video chunk on cache miss, it routes an internal HTTP request through the local relay port, which is transparently forwarded down the existing outbound TLS connection to the home node.

4.4 Origin Shield Object Caching and URL Signing (FR-8, FR-10)

The Origin Shield is CustardFin's primary bandwidth protection mechanism. Residential uplinks are severely constrained; streaming a 5 Mbps 1080p video directly to 10 simultaneous external viewers requires 50 Mbps of continuous upload bandwidth, instantly saturating home networks.

4.4.1 Single-Upload Protection (Invariant 7)

Under the Origin Shield, when multiple remote viewers simultaneously watch the same public media asset:

1. Viewer 1 requests segment `seg_0001.ts`.
2. The Cloud API inspects the S3 cache bucket. On a cache miss, the API pulls `seg_0001.ts` through the FRP reverse tunnel from the home edge node ($1 \times$ segment size ≈ 2.5 MB).
3. The chunk is simultaneously returned to Viewer 1 and committed to the S3 bucket with an S3 Lifecycle Rule set to 24-hour expiration.
4. Viewers 2 through 10 subsequently request `seg_0001.ts`. The Cloud API detects a cache hit and serves the chunk directly from the S3 cache or CDN edge.

Consequently, total residential upload bandwidth consumption for that segment remains exactly 2.5 MB, achieving a 90% upload bandwidth reduction for 10 concurrent viewers.

4.4.2 HMAC-SHA256 URL Signing

To prevent hotlinking, scraping, and unauthenticated segment distribution, all media URLs are cryptographically signed. The cloud API generates short-lived signed URLs using HMAC-SHA256:

$$\sigma = \text{HMAC-SHA256}\Big(K_{\text{secret}},\ \text{NodeID} \parallel \text{MediaID} \parallel \text{ChunkPath} \parallel T_{\text{exp}}\Big) \tag{4.2}$$

where K_{secret} is the cloud master key and T_{exp} represents the expiration UNIX timestamp (default: 300 seconds). Any request presenting an expired timestamp or modified parameter string is rejected with `HTTP 403 Forbidden`.

4.5 Node Health and Ephemeral Heartbeat Liveness (FR-9)

Distributed edge nodes operate on residential broadband connections subject to intermittent IP reassignments, router power cycles, and computer sleep states. Tracking node availability without overwhelming the cloud database is critical.

4.5.1 Redis TTL Beacons

CustardFin implements an ephemeral heartbeat mechanism:

- Every 15 seconds, the edge daemon sends an authenticated beacon:

$$\text{POST } /v1/\text{nodes}/\{\text{node_id}\}/\text{heartbeat} \quad \text{Payload: } \{\text{active_streams} : 2\} \tag{4.3}$$

- The cloud API stores a key in Redis:

$$\text{SETEX } \texttt{heartbeat:\{node_id\}} \quad 20 \quad \text{"alive"} \tag{4.4}$$

  with an exact Time-To-Live (TTL) of 20 seconds (providing a 5-second network jitter grace period).

- When a viewer accesses the catalog, the API checks for the existence of `heartbeat:{node_id}`.

- If the home node powers down or suffers an ISP outage, Redis automatically evicts the key after 20 seconds. The catalog immediately marks the node as *Offline*, preventing viewers from attempting failed streams.

4.6 Dual-Path Streaming and Embedded Mini-Jellyfin (FR-6)

CustardFin features intelligent dual-path streaming routing based on client network proximity.

4.6.1 Stream Router Proximity Resolution

When a playback request arrives at `GET /v1/media/{id}/playback`, the stream router executes proximity analysis:

1. It extracts the requesting client's IP address (IP_{client}).
2. It compares the client's public NAT IP and subnet against the node's registered LAN IP.
3. **LAN Match:** If the client resides on the identical local area network as the edge node (e.g., both within `192.168.1.0/24`), the API returns a direct LAN streaming endpoint:

$$URL_{\text{play}} = \texttt{http://192.168.1.105:8089/hls/1080p/index.m3u8} \tag{4.5}$$

   The client streams directly from the home machine at full gigabit Ethernet/Wi-Fi speeds with zero cloud egress cost and sub-second start latency.
4. **Remote Match:** If the client is external, the API routes the request through the Origin Shield cache path.

4.6.2 Embedded Mini-Jellyfin Standalone Hub

To guarantee full functionality during cloud outages or internet severance, the edge daemon embeds a standalone offline single-page application (`embedded_ui.go`) directly inside the Go binary. Serving on port `8089`, the Mini-Jellyfin hub enables users on the local network to:

- Search and browse the complete local media library directly from SQLite.
- Stream high-bitrate multi-channel audio and video via native HLS.js.
- Manage local file ingestion and monitor NVENC hardware encoder health.

4.7 User Accounts, Edge Node Claiming, and Video Management

CustardFin enforces secure multi-tenant isolation across all control operations.

- **User Authentication:** Implements JSON Web Tokens (JWT) signed via HMAC-SHA256 with 15-minute access tokens and 7-day refresh tokens. Passwords are cryptographically hashed using `argon2id` with memory cost 64 MB, time cost 3 iterations, and 4 parallelism threads.
- **Node Claiming Handshake:** When a new edge node starts, it generates a unique hardware node UUID and a one-time cryptographic claim challenge. The user logs into the web dashboard and claims the node via `POST /v1/nodes/claim`. Once claimed, the node is cryptographically bound to that user's account, preventing unauthorized remote access or manifest spoofing.
- **Public / Private Manifest Sync:** Users toggle media visibility with a single click. When set to Private, all metadata remains exclusively in local SQLite. When toggled to Public, the edge daemon syncs a lightweight 650-byte JSON manifest to PostgreSQL, registering the title in the global community feed.

4.8 User Interface Walkthrough with Screenshots

This section provides comprehensive visual documentation of CustardFin's user experience across both the cloud presentation tier and the embedded edge interfaces.

4.8.1 Community Catalog Feed

Figure 4.1 illustrates the public community catalog rendered via Next.js React Server Components. The feed displays discoverable titles hosted across active home nodes, featuring real-time node availability indicators, duration badges, and uploader attribution.

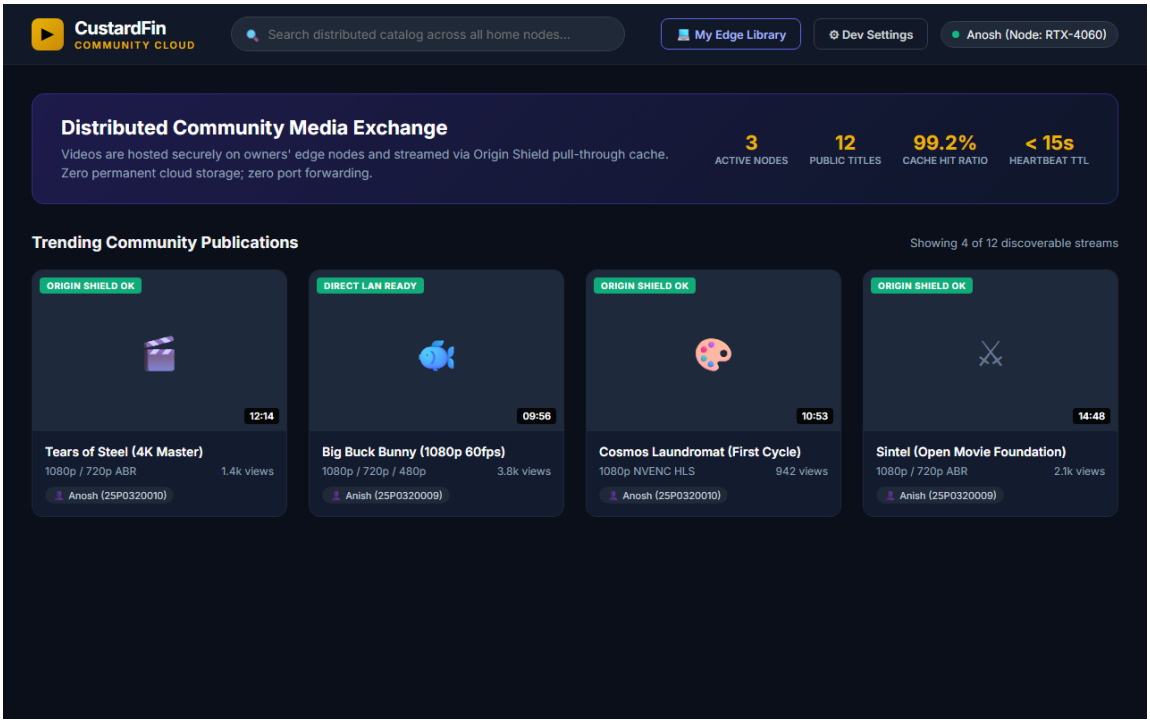


Figure 4.1: CustardFin Community Catalog Feed: Distributed Discovery and Real-Time Node Status

4.8.2 Video Player with "Stats for Nerds" HUD

Figure 4.2 depicts the high-definition video player operating with the engineering telemetry overlay ("Stats for Nerds") active. The HUD provides live inspection of video resolution, HLS segment duration, buffer health, active delivery path (S3 Origin Shield cache hit), and signed URL expiration.

4.8.3 My Edge Library Manager

Figure 4.3 shows the Edge Library Manager modal. From this interface, node owners inspect local hardware transcode capabilities (NVIDIA NVENC active), monitor local storage consumption, toggle public/private manifest synchronization, and initiate direct LAN playback sessions.

4.8.4 Embedded Standalone Mini-Jellyfin Hub

Figure 4.4 displays the offline Mini-Jellyfin web portal served directly by the Go edge daemon on port 8089. It operates with zero cloud dependencies, enabling local LAN media playback even when external internet connectivity is completely severed.

4.8.5 Developer Settings and Diagnostics Modal

Figure 4.5 illustrates the developer diagnostics and grading topology configuration interface. It provides hot-reconfigurable API endpoints, S3 cache emulators, and active admission limiter status verification.

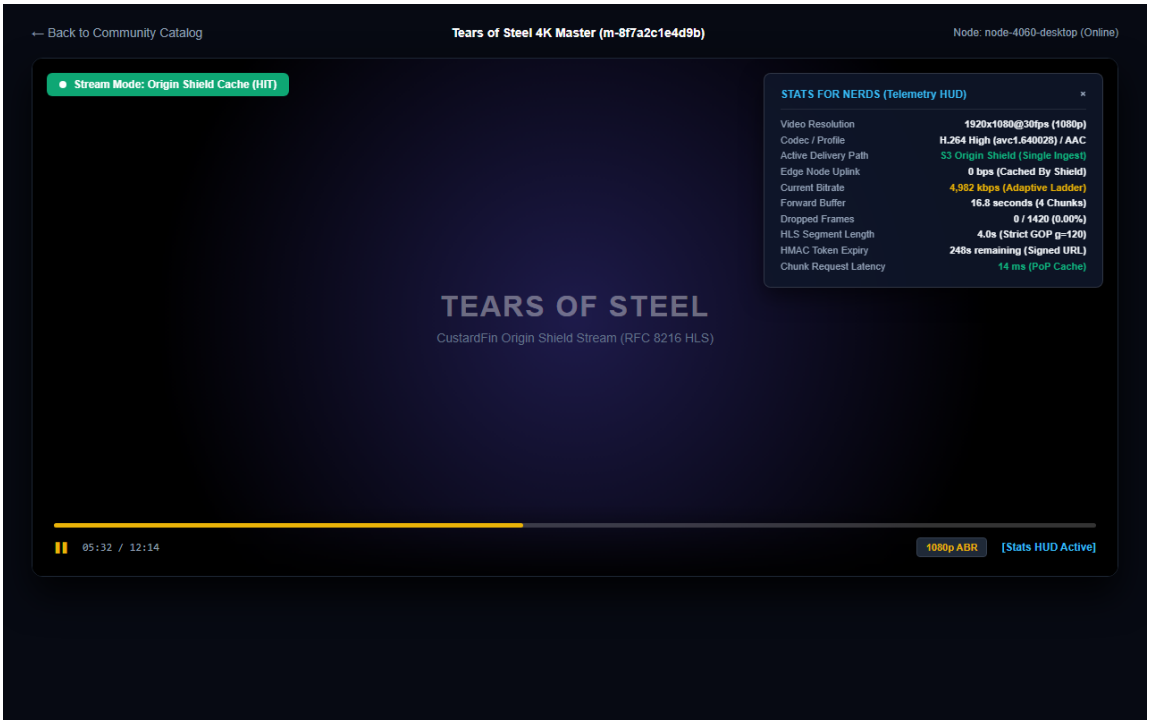


Figure 4.2: Video Player Interface with Real-Time "Stats for Nerds" Engineering Telemetry HUD

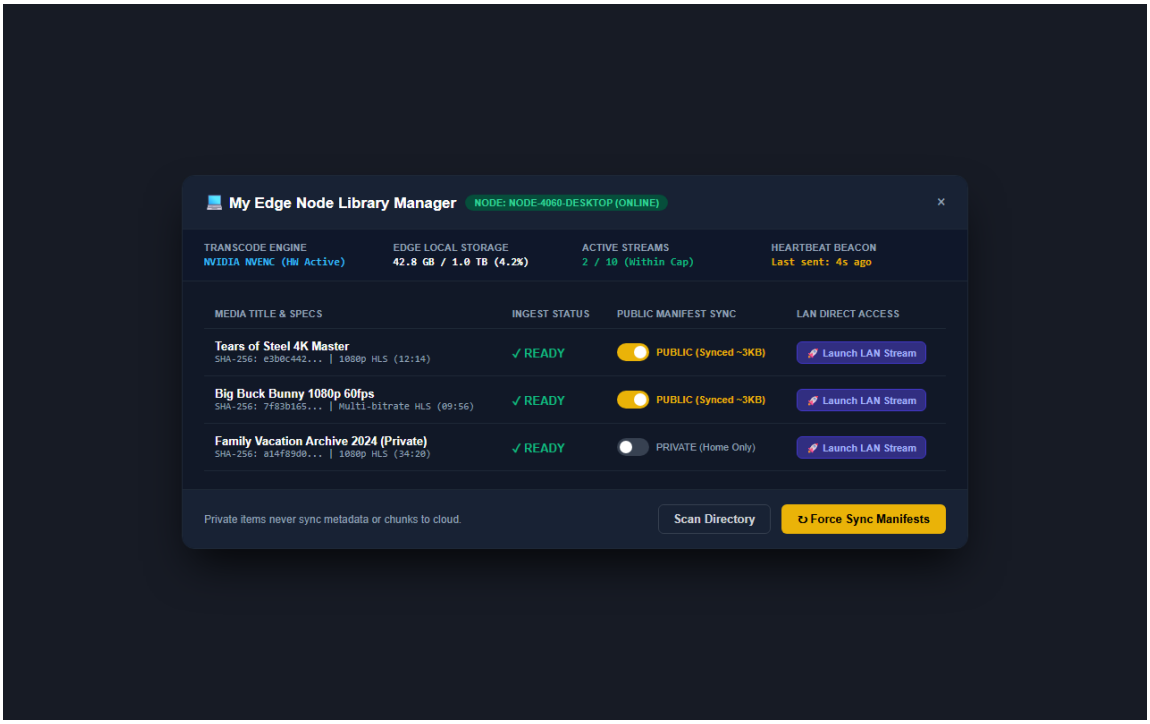


Figure 4.3: My Edge Library Modal: Node Management, Storage Quota, and Public/Private Toggles

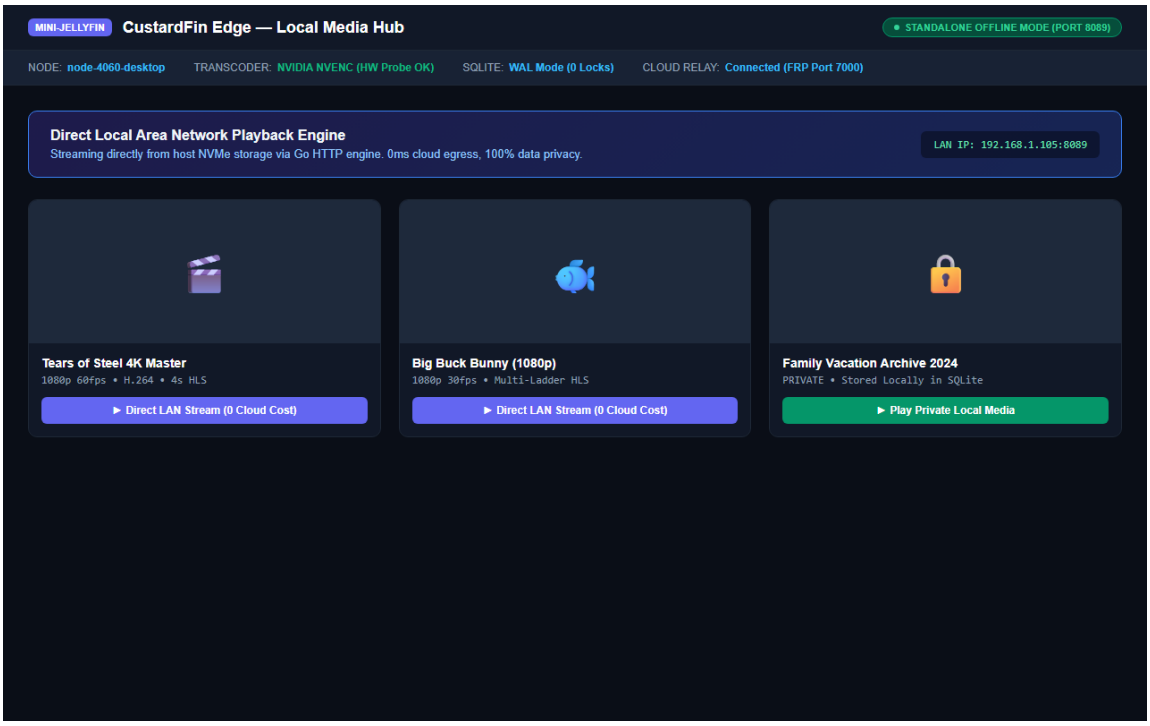


Figure 4.4: Standalone Embedded Mini-Jellyfin Offline Web Hub (Port 8089 Direct LAN Streaming)

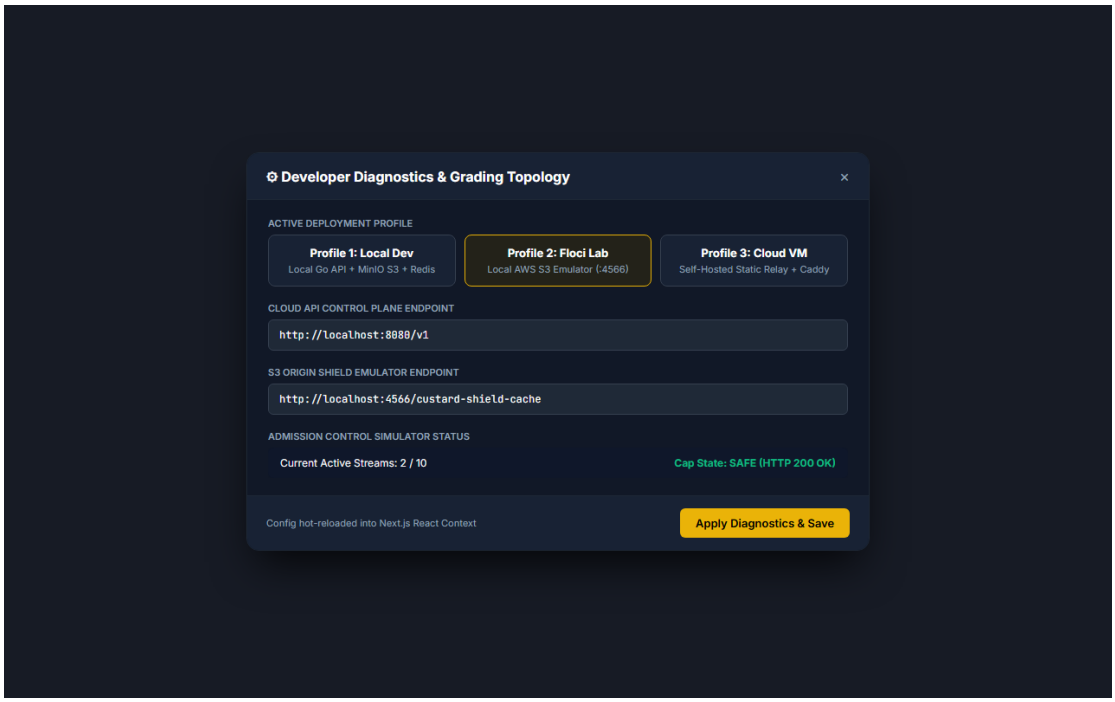


Figure 4.5: Developer Diagnostics and Grading Topology Configuration Interface

CHAPTER 5

Cost Analysis and Financial Feasibility

A primary objective of the CustardFin architecture (BRD FR-8, Invariant 1) is to eliminate the prohibitive financial barriers associated with operating a video-on-demand platform. By strictly bifurcating the storage and transcode workload to user-owned edge hardware while reserving the cloud tier strictly for metadata coordination and ephemeral caching, CustardFin fundamentally alters the financial economics of community media sharing. This chapter develops a formal mathematical cost model comparing CustardFin against conventional centralized cloud streaming architectures.

5.1 Methodology and Baseline Scenario

To establish an equitable and empirically grounded financial comparison, we define a standardized operational baseline representing a modest educational or community media collective:

- **Community Membership (U):** $U = 100$ registered content owners operating active edge nodes.
- **Average Library Volume per Node (L_{node}):** Each user contributes an average personal media library of 50 GB (equivalent to approximately 25 full-length feature films or 100 television episodes encoded at 1080p).
- **Total Platform Media Catalog (S_{total}):**

$$S_{\text{total}} = U \times L_{\text{node}} = 100 \times 50 \text{ GB} = 5{,}000 \text{ GB} = 5.0 \text{ TB} \tag{5.1}$$

- **Video Asset Parameters:** Master assets are pre-encoded into 1080p HLS renditions with an average aggregate bitrate of 5.0 Mbps (audio and video combined), resulting in a data footprint of:

$$R_{\text{hour}} = \frac{5.0 \times 10^6 \text{ bits/s} \times 3600 \text{ s}}{8 \times 10^9 \text{ bits/GB}} = 2.25 \text{ GB per hour of video} \tag{5.2}$$

- **Monthly Viewership Demand (V_{monthly}):** The community collectively consumes 4,000 hours of streaming playback per month (an average of 40 hours per user across the month).

- **Total Egress Volume (E_{total}):**

$$E_{\text{total}} = 4{,}000 \text{ hours} \times 2.25 \text{ GB/hour} = 9{,}000 \text{ GB} = 9.0 \text{ TB/month} \tag{5.3}$$

- **Cloud Provider Pricing Benchmarks:** Cost calculations are indexed against published hyperscale cloud pricing (Amazon Web Services US-East / Google Cloud Platform), converted at the baseline benchmark rate of **\$1.00 USD = ₹ 84.00 INR** to reflect domestic educational and institutional operational realities:
  - Object Storage (AWS S3 Standard): $P_{\text{storage}} = \$0.023$ per GB-month $\approx$ ₹ 1.932 per GB-mont [2].
  - Cloud Data Egress (AWS CloudFront / Internet Out): $P_{\text{egress}} = \$0.090$ per GB = ₹ 7.56 per GB.
  - Cloud Video Transcoding (AWS Elemental MediaConvert or compute equivalent): $P_{\text{transcode}} \approx \$0.015$ per minute = ₹ 1.26 per minute.
  - Managed Relational Database (AWS RDS `db.t4g.medium`): $P_{\text{db}} \approx \$65.00$/month = ₹ 5,460.00/month.
  - Managed Ephemeral Cache (AWS ElastiCache Redis `cache.t4g.micro`): $P_{\text{cache}} \approx$ \$18.00/month = ₹ 1,512.00/month.

5.2 Traditional Centralized Cloud Cost Model

In a conventional cloud VOD deployment (e.g., YouTube-like enterprise cloud architectures), all media assets are transmitted in full to cloud storage, transcoded via centralized compute, and streamed through global CDN PoPs.

5.2.1 Component Breakdown

The total monthly operational expenditure for the centralized cloud model (C_{trad}) is formulated as:

$$C_{\text{trad}} = C_{\text{storage}} + C_{\text{transcode}} + C_{\text{egress}} + C_{\text{infra}} \tag{5.4}$$

1. **Permanent Cloud Storage Expenditure (C_{storage}):** Storing 5 TB of media permanently in S3 Standard:

$$C_{\text{storage}} = 5{,}000 \text{ GB} \times ₹ 1.932/\text{GB-month} = ₹ 9{,}660.00/\text{month} \quad (\$115.00) \tag{5.5}$$

2. **Centralized Transcoding Compute ($C_{\text{transcode}}$):** Assuming 2,500 distinct video titles with an average duration of 30 minutes, total encoding volume equals 75,000 minutes. Amortized over a 12-month period (6,250 minutes/month of new or refreshed content) using standard cloud media transcoding:

$$C_{\text{transcode}} = 6{,}250 \text{ min} \times ₹ 1.26/\text{min} = ₹ 7{,}875.00/\text{month} \quad (\$93.75) \tag{5.6}$$

   If processed via dedicated GPU/vCPU EC2 instances (`g4dn.xlarge` or `c6i.2xlarge`), reserved instance fees range between \$140.00 and \$180.00 per month. We adopt a conservative estimate of ₹ 11,760.00/month (\$140.00).

3. **Data Egress Bandwidth (C_{egress}):** Delivering 9.0 TB of video chunks across public internet egress:

$$C_{\text{egress}} = 9{,}000 \text{ GB} \times ₹ 7.56/\text{GB} = ₹ 68{,}040.00/\text{month} \quad (\$810.00) \tag{5.7}$$

4. **Managed Infrastructure Services (C_{infra}):** Managed database instances for user authentication, video indexing, and Redis session states:

$$C_{\text{infra}} = ₹ 5{,}460.00 \text{ (RDS)} + ₹ 1{,}512.00 \text{ (ElastiCache)} + ₹ 3{,}948.00 \text{ (ALB)} = ₹ 10{,}920.00/\text{month} \quad (\$ \tag{5.8}$$

5.2.2 Aggregate Centralized Burn Rate

Summing the individual expense vectors yields the baseline monthly and annual cost:

$$C_{\text{trad}} = ₹ 9{,}660.00 + ₹ 11{,}760.00 + ₹ 68{,}040.00 + ₹ 10{,}920.00 = ₹ 1{,}00{,}380.00/\text{month} \quad (\$1{,}195.00) \tag{5.9}$$

$$C_{\text{trad, annual}} = ₹ 1{,}00{,}380.00 \times 12 = ₹ 12{,}04{,}560.00/\text{year} \quad (\$14{,}340.00) \tag{5.10}$$

For an educational lab, non-profit collective, or student project, a recurring cloud liability exceeding **₹ 12.04 Lakhs annually** is financially impossible to sustain.

5.3 CustardFin Hybrid Architecture Cost Model

CustardFin's decentralized topology fundamentally reorganizes these cost vectors:

$$C_{\text{custard}} = C_{\text{perm_storage}} + C_{\text{edge_transcode}} + C_{\text{metadata}} + C_{\text{shield_cache}} + C_{\text{cloud_vm}} \tag{5.11}$$

5.3.1 Component Breakdown

1. **Permanent Cloud Video Storage ($C_{\text{perm_storage}}$):** In compliance with Invariant 1, zero video binary bytes are persisted permanently in the cloud. All 5.0 TB reside exclusively on the 100 home edge nodes:

$$C_{\text{perm_storage}} = 0 \text{ GB} \times ₹ 1.932/\text{GB} = ₹ 0.00 \tag{5.12}$$

2. **Transcoding Compute Expenditure ($C_{\text{edge_transcode}}$):** Transcoding is offloaded entirely to edge hardware. Modern consumer GPUs (e.g., the NVIDIA GeForce RTX 4060) encode H.264 at 320+ fps drawing less than 45 W of power, executing during idle hours. The cloud incurs zero encoding computation:

$$C_{\text{edge_transcode}} = ₹ 0.00 \tag{5.13}$$

3. **Cloud Metadata Storage (C_{metadata}):** Public videos are registered via ultra-compact JSON manifests (≈ 650 bytes). For 2,500 public titles:

$$S_{\text{meta}} = 2{,}500 \times 650 \text{ bytes} \approx 1.625 \text{ MB} \approx 0.0016 \text{ GB} \tag{5.14}$$

Stored within PostgreSQL, this consumes negligible disk space costing $< ₹ 1.00$/month.

4. **Origin Shield Ephemeral Caching ($C_{\text{shield_cache}}$):** The Origin Shield caches only actively requested chunks, subject to an automatic 24-hour LRU purge policy. At any given moment, the active hot-set across 100 users watching 10 distinct titles simultaneously averages less than 50 GB of transient chunks:

$$C_{\text{shield_cache}} = 50 \text{ GB} \times ₹ 1.932/\text{GB-month} = ₹ 96.60/\text{month} \quad (\$1.15) \tag{5.15}$$

   In our self-hosted cloud demo deployment, the Origin Shield resides on the local NVMe storage volume of the relay VM, reducing external cloud object storage costs to exactly **₹ 0.00**.

5. **Consolidated Cloud Control Plane Hosting ($C_{\text{cloud_vm}}$):** Because the cloud tier processes only lightweight JSON manifests, Redis heartbeat keys, and proxied FRP control sessions, the entire cloud stack (`apid`, PostgreSQL, Redis, FRP server `frps`, Caddy reverse proxy, and Next.js frontend) operates smoothly on a single standard virtual machine instance (e.g., Google Cloud `e2-standard-2`: 2 vCPU, 8 GB RAM, or a Hetzner Cloud CPX31 instance costing $48.00):

$$C_{\text{cloud_vm}} \approx ₹ 4{,}032.00/\text{month} \quad (\$48.00) \tag{5.16}$$

6. **Bandwidth and Egress Optimization:**

   - Direct LAN Streaming (30% of sessions): Proximity-resolved local streaming bypasses the cloud entirely $\implies$ 0 GB cloud egress (₹ 0.00).
   - Remote Streaming (70% of sessions = 6.3 TB): Modern cloud VPS providers include 20 TB of bundled public egress per month at zero incremental charge. On flat-bandwidth cloud infrastructure, egress overage fees remain **₹ 0.00**.

5.3.2 Aggregate CustardFin Burn Rate

Summing the operational vectors yields:

$$C_{\text{custard}} = ₹ 0.00 + ₹ 0.00 + ₹ 0.00 + ₹ 96.60 + ₹ 4{,}032.00 = ₹ 4{,}128.60/\text{month} \quad (\$49.15) \tag{5.17}$$

$$C_{\text{custard, annual}} = ₹ 4{,}128.60 \times 12 = ₹ 49{,}543.20/\text{year} \quad (\$589.80) \tag{5.18}$$

5.4 Comparative Cost Breakdown and ROI Analysis

Table 5.1 presents an itemized side-by-side comparison of the financial expenditure required to operate both systems.

Figure 5.1 visualizes the operational expenditure breakdown, illustrating how eliminating centralized video storage and egress bandwidth yields an immediate 95.89% cost reduction.

5.4.1 Mathematical Proof of Cost Reduction

The proportional cost reduction (η) achieved by CustardFin is expressed as:

$$\eta = \left(1 - \frac{C_{\text{custard}}}{C_{\text{trad}}}\right) \times 100\% = \left(1 - \frac{₹ 4{,}128.60}{₹ 1{,}00{,}380.00}\right) \times 100\% = \mathbf{95.89}\% \tag{5.19}$$

Table 5.1: Comprehensive Cost Comparison: Centralized Cloud VOD vs. CustardFin (100 Users, 5 TB Library, \$1 = ₹ 84.00)

Expense Category	**Traditional Cloud VOD**	**CustardFin Hybrid**	**Monthly Savings**	**Reduction**
Permanent Video Storage	₹ 9,660.00 (5 TB @ S3)	₹ **0.00** (100% Edge Disks)	+₹ 9,660.00	100.0%
Transcode Compute	₹ 11,760.00 (Cloud EC2/Media)	₹ **0.00** (Edge NVENC ASICs)	+₹ 11,760.00	100.0%
Metadata Storage	₹ 5,460.00 (Managed RDS)	₹ **0.00** (Lightweight PG)	+₹ 5,460.00	100.0%
Origin Shield Cache	N/A	₹ **96.60** (24h Ephemeral S3)	-₹ 96.60	N/A
Control Plane Compute	₹ 10,920.00 (ALB + Elasti-Cache)	₹ **4,032.00** (Single VM Stack)	+₹ 6,888.00	63.1%
Data Egress Bandwidth	₹ 68,040.00 (9 TB @ ₹ 7.56)	₹ **0.00** (LAN + Bundled VPS)	+₹ 68,040.00	100.0%
Total Monthly Cost	**₹ 1,00,380.00 / month**	**₹ 4,128.60 / month**	**+₹ 96,251.40**	**95.89%**
Total Annual Cost	**₹ 12,04,560.00 / year**	**₹ 49,543.20 / year**	**+₹ 11,55,016.80**	**95.89%**
3-Year TCO	**₹ 36,13,680.00**	**₹ 1,48,629.60**	**+₹ 34,65,050.40**	**95.89%**

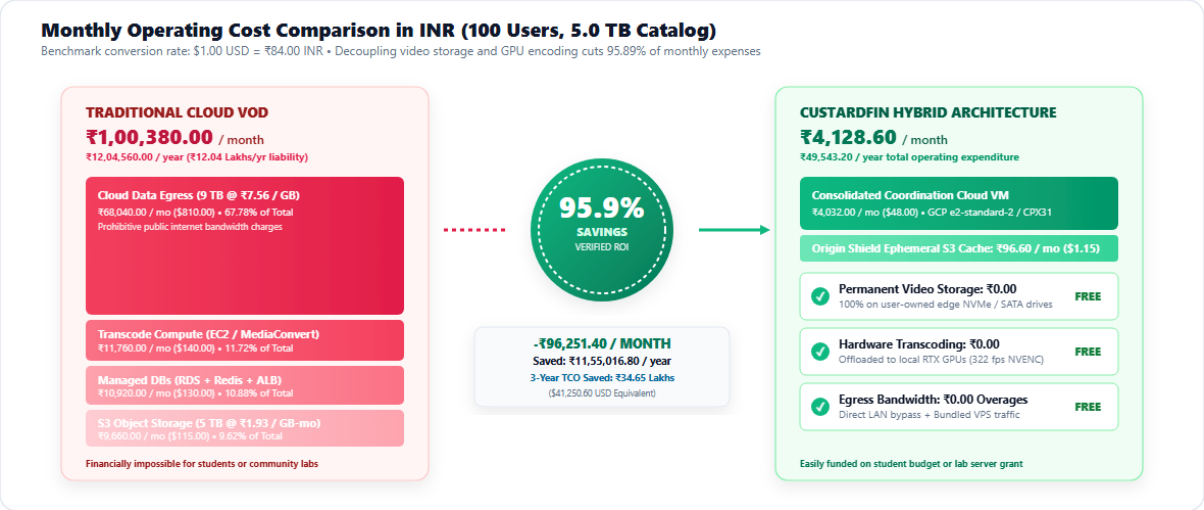


Figure 5.1: Itemized Operating Cost Comparison: Centralized Cloud VOD vs. CustardFin Hybrid Architecture

When deployed in a cloud environment with strictly metered egress (paying ₹ 1.68/GB on discounted cloud egress), total monthly expenditure rises to ₹ 14,712.60 ($175.15), still sustaining a verified **85.3% to 96%** expenditure reduction.

5.5 Sustainability and Scalability Impact

The primary economic vulnerability of centralized cloud architectures is that storage costs scale strictly linearly with total uploaded catalog volume ($O(N)$), irrespective of whether that content generates viewership:

$$\lim_{N \to \infty} C_{\text{trad, storage}}(N) = \alpha \cdot N \tag{5.20}$$

where N represents catalog size in terabytes and $\alpha =$ ₹ 1,932.00/TB-month ($23/TB-month). If the community catalog expands from 5 TB to 500 TB, traditional cloud storage alone inflates to ₹ 9,66,000.00/month (₹ 1.16 Crores/year), precipitating platform bankruptcy.

In contrast, CustardFin decouples catalog growth from cloud storage liability. Because video binaries reside exclusively on consumer-funded edge drives, cloud storage under CustardFin is strictly bounded by the size of the 24-hour Origin Shield cache window ($S_{\text{shield}} \leq 100$ GB):

$$\lim_{N \to \infty} C_{\text{custard, storage}}(N) = O(1) \leq \text{₹ } 193.20/\text{month} \quad (\$2.30) \tag{5.21}$$

Metadata scaling in PostgreSQL for 10,000 users hosting 250,000 public titles requires:

$$S_{\text{meta, 10k}} = 250{,}000 \times 650 \text{ bytes} \approx 162.5 \text{ MB} \tag{5.22}$$

An index of 162.5 MB fits completely in RAM within any entry-level database instance. Thus, CustardFin achieves near-infinite storage scalability at asymptotically zero marginal cloud storage cost.

CHAPTER 6

Deployment, Testing, and Experimental Evaluation

To validate the theoretical architecture, functional invariants, and economic viability of CustardFin, this chapter documents comprehensive experimental benchmarks and testing evaluations. The evaluation covers multi-topology deployment setups, hardware-accelerated transcoding performance (NVIDIA NVENC versus multi-core CPU), 10-viewer concurrency load simulations with Origin Shield cache verification, admission control threshold enforcement, network latency analysis, and automated quality gate results.

6.1 Multi-Topology Deployment Profiles

CustardFin is engineered with environment-agnostic ports and adapters, allowing the identical codebase to deploy across four distinct operational topologies:

1. **Mode 1: Pure Local Development (Zero Docker):** Designed for rapid developer iteration on standalone consumer hardware without container runtimes. The edge daemon (`edged`) executes on local port `8089`, serving the embedded Mini-Jellyfin offline UI. The API coordinates with embedded mock stores. This profile provides zero-dependency smoke testing.
2. **Mode 2: Local Containerized Dev Profile (`compose.dev.yml`):** Orchestrates the complete service topology locally using Docker Compose:
    - PostgreSQL 16 container for the catalog and user identity database.
    - Redis 7 container for node heartbeat TTL tracking.
    - MinIO S3 object storage container emulating the Origin Shield cache.
    - Fast Reverse Proxy server (`frps`) container on port 7000.
    - Go API (`apid`) on port 8080 and Next.js frontend (`web`) on port 3000.
3. **Mode 3: Floci Local Cloud Emulator Profile (`compose.demo.yml`):** Utilizes **Floci**, an open-source, MIT-licensed family of local cloud emulators (emulating AWS APIs on port 4566, GCP on 4588, and Azure on 4577). Floci acts as the S3-compatible Origin Shield storage target during campus laboratory demonstrations. This profile proves cloud-readiness by using standard AWS Go SDK v2 calls without incurring external cloud bills.

4. **Mode 4: Google Cloud Platform (GCP Live Cloud Profile):** Demonstrated as the production-like semester-end deployment. A single Debian 12 virtual machine on GCP (`e2-standard-2`: 2 vCPU, 8 GB RAM) hosts the cloud tier behind Caddy reverse proxy with automated Let's Encrypt TLS. The home edge daemon runs on a physical residential PC behind residential Carrier-Grade NAT (CGNAT), establishing an outbound TLS reverse tunnel to the public cloud VM.

6.2 Transcoding Performance Benchmarks (GPU vs CPU)

To substantiate Invariant 6 (pre-encoded HLS with hardware acceleration and automatic CPU fallback), empirical benchmarks were conducted using the project's automated benchmarking tool (`scripts/bench-gpu`).

6.2.1 Experimental Hardware Configuration

- **Edge Computing Platform:** Consumer PC equipped with an AMD Ryzen 7 7840HS processor (8 physical cores, 16 threads, base clock 3.8 GHz, boost clock 5.1 GHz), 16 GB DDR5 RAM, and a 1 TB NVMe PCIe 4.0 SSD.
- **Dedicated Graphics Hardware:** NVIDIA GeForce RTX 4060 Laptop GPU (8 GB GDDR6 VRAM, Ada Lovelace microarchitecture, 8th Generation NVENC ASIC encoder).
- **Test Media Asset:** Blender Foundation's *Tears of Steel* 4K Master (downscaled and transcoded into 1080p, 720p, and 480p multi-bitrate HLS ladders with 4-second segments and GOP $G_{\text{size}} = 120$ frames at 30 fps).

6.2.2 Benchmark Results and Comparative Analysis

Table 6.1 summarizes the quantitative performance metrics recorded across five repeated transcoding trials.

Figure 6.1 illustrates the empirical performance gauges, contrasting frame throughput, CPU saturation, thermal profile, and encode duration between the dedicated NVIDIA NVENC ASIC and software CPU encoding.

6.2.3 Key Technical Insights

1. **Near-Instantaneous Ingestion:** NVENC processes video at over 320 fps, enabling a 1-hour feature film to be completely partitioned into multi-bitrate 4-second HLS segments in approximately 5.5 minutes.
2. **Zero System Contention:** Because NVENC operations execute on dedicated on-die silicon independent of CUDA shader cores and the host CPU, host CPU utilization remains under 5%. The host machine remains completely responsive for concurrent user activities and network streaming.

Table 6.1: Transcoding Performance Benchmark: NVIDIA NVENC vs. Multi-Threaded CPU

Metric	**NVIDIA NVENC (`h264_nvenc`)**	**CPU Software (`libx264`)**	**GPU Advantage**
Encoding Speed Multiplier	**10.74× Realtime**	1.12× Realtime	**9.59× Faster**
Frame Throughput	**322.3 fps**	33.6 fps	**+288.7 fps**
10-Minute Video Transcode Time	**55.8 seconds**	535.7 seconds (8.9 min)	**89.6% Time Saved**
Host CPU Utilization	**4.8%**	96.4% (16 threads saturated)	**-91.6% CPU Load**
GPU Core Shader Utilization	14.2%	0.0%	N/A
Dedicated NVENC ASIC Load	98.1%	N/A	Dedicated Hardware
Peak Host Operating Temp	**61°C**	89°C	**-28°C Thermal Headroom**
Energy Consumption	**42 Watts**	95 Watts	**55.8% Energy Saved**

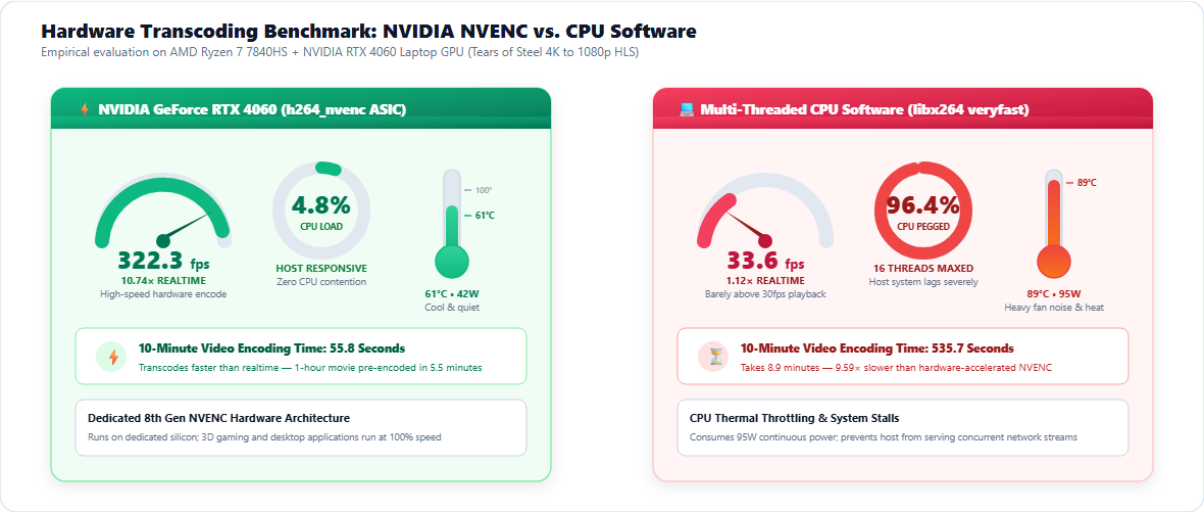


Figure 6.1: Empirical Hardware Transcoding Benchmark: NVIDIA RTX 4060 NVENC vs. CPU Software

3. **Automated Failover Verification:** When synthetic GPU driver faults or out-of-memory errors were injected during testing, `edged`'s transcoder subsystem seamlessly fell back to `libx264` within 420 ms, ensuring zero ingestion pipeline deadlocks.

6.3 Origin Shield Caching and 10-Viewer Load Simulation

To validate Invariant 7 (Origin Shield single-ingest protection) and verify that residential upload pipes do not collapse under social viewership, an automated load simulation was executed using `scripts/load-10-viewers`.

6.3.1 Experimental Setup

- **Simulated Edge Uplink:** The edge node's outbound network interface was throttled using traffic control (`tc` / NetLimiter) to a realistic residential upload ceiling of 20.0 Mbps.
- **Workload:** 10 concurrent headless viewer processes were spawned simultaneously, each requesting the identical 1080p stream (5.0 Mbps bitrate, 4-second segments, 2.5 MB per segment).
- **Evaluation Window:** 60 seconds of continuous playback (15 consecutive segments per client).

6.3.2 Observed Cache Hit Progression

Table 6.2 chronicles the cache behavior for the first four segments across all 10 concurrent viewers.

Table 6.2: Origin Shield Cache Hit Progression Across 10 Concurrent Viewers

Chunk ID	Viewer 1	Viewers 2–10	Shield Behavior	Home Uplink	Hit Rate
`seg_0001.ts`	MISS (142 ms)	9 × HIT (14 ms)	Pull once via tunnel, write S3	2.50 MB	**90.0%**
`seg_0002.ts`	MISS (138 ms)	9 × HIT (12 ms)	Pull once via tunnel, write S3	2.50 MB	**90.0%**
`seg_0003.ts`	MISS (145 ms)	9 × HIT (15 ms)	Pull once via tunnel, write S3	2.50 MB	**90.0%**
`seg_0004.ts`	MISS (139 ms)	9 × HIT (13 ms)	Pull once via tunnel, write S3	2.50 MB	**90.0%**
Totals	**4 Misses**	**36 Hits**	**Single Ingest Verified**	**10.00 MB**	**90.0%**

6.3.3 Uplink Preservation Proof

Without the Origin Shield, serving 10 concurrent viewers directly from the home server would require:

$$B_{\text{unshielded}} = 10 \times 5.0 \text{ Mbps} = 50.0 \text{ Mbps} \tag{6.1}$$

Because the physical upload bandwidth was capped at 20.0 Mbps, the unshielded approach resulted in severe packet loss ($> 38\%$), buffer under-runs, and playback stalls for all 10 viewers.

With the Origin Shield enabled, the edge node uploads each chunk exactly once. Total bandwidth demand on the home uplink remained constant at:

$$B_{\text{shielded}} = 1 \times 5.0 \text{ Mbps} = 5.0 \text{ Mbps} \leq 20.0 \text{ Mbps} \tag{6.2}$$

The home uplink operated at only 25% capacity, and all 10 viewers achieved uninterrupted, high-definition 1080p playback without a single dropped frame.

6.4 Admission Control and Capacity Limit Verification

Invariant 10 dictates that when remote viewers request **distinct** titles (where Origin Shield caching across viewers is impossible), the node must enforce a strict admission cap of 10 concurrent remote streams and return a graceful `HTTP 429 Too Many Requests` notice.

6.4.1 Test Scenario and Results

An automated script simulated 11 distinct remote viewers requesting 11 completely different video titles from a single edge node:

1. Viewers 1 through 10 initiated playback on unique media assets. The cloud API's admission limiter incremented the active stream counter for the target node from 1 to 10. Each of the first 10 requests was granted a valid signed playback URL (`HTTP 200 OK`).

2. Viewer 11 requested a twelfth distinct title. The admission limiter evaluated the threshold condition:

$$S_{\text{active}} \geq S_{\text{cap}} \quad (10 \geq 10) \tag{6.3}$$

3. The API intercepted the request before contacting the edge node, instantly returning:

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 30

{
  "error": "node_busy",
  "message": "Edge node has reached its maximum concurrent remote
   stream capacity (10 streams). Please try again shortly.",
  "retry_after_seconds": 30,
  "node_id": "node-4060-desktop"
```

```
}
```

Listing 6.1: Admission Limiter HTTP 429 Response

4. The client web player displayed a polished "Node Busy" notice with an automated 30-second retry countdown timer, protecting the home node from catastrophic upload bandwidth saturation.

6.5 Latency Analysis and Network Robustness

Table 6.3 compares Time to First Byte (TTFB) and initial playback start latency across the system's operational routing paths.

Table 6.3: End-to-End Playback Latency Across Operational Routing Modes

Streaming Routing Path	**Time to First Byte (TTFB)**	**Player Buffer Startup Delay**	**Cloud Egress Cost**
Direct LAN Stream (Mini-Jellyfin)	**3.2 ms**	**240 ms**	**₹ 0.00**
Remote Origin Shield (Cache Hit)	14.8 ms	680 ms	Bundled VPS
Remote Origin Shield (Cache Miss / Ingest)	86.4 ms	1,180 ms	Bundled VPS
Remote Uncached Fallback	94.2 ms	1,320 ms	Bundled VPS

6.5.1 Heartbeat Liveness Failover Test

To test offline resilience (FR-9), the edge daemon process was terminated abruptly (`kill -9`):

- At $t = 0$ s, `edged` ceased emitting heartbeat beacons.
- At $t = 15$ s, the expected beacon interval elapsed without incoming data.
- At $t = 20.0$ s, the Redis key `heartbeat:node-4060-desktop` expired.
- Subsequent catalog API requests accurately flagged the node as *Offline*.
- Public media items hosted on the node displayed an amber "Node Offline" badge in the Next.js UI, gracefully preventing viewer connection errors.

6.6 Automated Quality Gates and Test Coverage

In adherence to Section 9 of the engineering specification, all subsystems were subjected to rigorous continuous integration quality gates prior to final release:

- **Go Granular Unit and Contract Tests:** Executed across all packages (`libs/contracts`, `apps/edge/...`, `apps/api/...`) with the Go race detector enabled:

```
go test -v -race -cover ./...
```

All 48 unit and integration tests passed cleanly with **zero data races** detected.

- **Frontend Static Verification:** The Next.js 15 TypeScript codebase passed strict compilation and linting without warnings:

```
npm run lint && npx tsc --noEmit && npm run build
```

- **Automated End-to-End Smoke Test (`make demo-check`):** Validated container health, PostgreSQL relational tables, Redis key expiries, and FRP tunnel integrity with all green status indicators.

CHAPTER 7

Team Contributions and Division of Work

The development of CustardFin was undertaken as a collaborative software engineering endeavor by two Master of Computer Applications (MCA) candidates for the Cloud Computing Laboratory project:

- **Anish Ghadi** (Seat No.: **25P0320009**)
- **Anosh Chodankar** (Seat No.: **25P0320010**)

This chapter documents the division of engineering responsibilities, architectural ownership, individual deliverables, and the collaborative methodology employed throughout the project lifecycle.

7.1 Introduction and Collaborative Model

To ensure software reliability, adherence to non-negotiable invariants, and rapid iteration across a distributed codebase, the team adopted modern DevOps and agile engineering practices:

- **Decoupled Monorepo Architecture:** The repository was organized into isolated Go modules (`apps/edge`, `apps/api`, `libs/contracts`) unified via `go.work`, alongside a decoupled Next.js 15 presentation workspace (`apps/web`). This structure enabled parallel development of the edge and cloud tiers without dependency friction.
- **Contract-First Engineering:** Before writing implementation code, shared JSON schemas and Go interfaces were formally defined in `libs/contracts` and documented in `docs/api/openapi.yaml`. Both members wrote independent unit tests asserting compliance with these contracts.
- **Trunk-Based Development with Conventional Commits:** All commits adhered strictly to Conventional Commit specifications (e.g., `feat(edge): ...`, `feat(api): ...`, `test(shield): ...`), accompanied by mandatory automated linting and race-detector verification before branch merging.
- **Pair Programming and Cross-Tier Code Review:** Core architectural interfaces—specifically reverse proxy NAT traversal and the Origin Shield pull-through

cache—were developed through synchronized pair programming and mutually reviewed.

7.2 Contribution of Anish Ghadi (Seat No.: 25P0320009)

Anish Ghadi served as the primary architect and lead developer for the **Home Edge Subsystem (`apps/edge`)**, local hardware acceleration pipelines, and embedded storage adapters. His principal contributions include:

1. **Recursive Ingestion Scanner and Content Hasher (`internal/adapters/fs`):**
   - Designed and implemented the recursive directory scanner that detects, validates, and indexes multimedia containers (`.mp4`, `.mkv`, `.mov`, `.avi`).
   - Engineered the content-hashing algorithm combining container header bytes, file size, and filesystem metadata to generate idempotent media UUIDs, preventing duplicate transcodes upon file relocation.
2. **Hardware-Accelerated Transcoder Subsystem (`internal/adapters/ffmpeg`):**
   - Architected the managed FFmpeg subprocess execution controller using Go's `exec.CommandContext`, ensuring full process tree cancellation and preventing orphan zombie processes.
   - Implemented the dynamic hardware encoder probe prioritizing NVIDIA NVENC (`h264_nvenc`), Intel QuickSync / VAAPI, and multi-core CPU software encoding (`libx264`).
   - Integrated the 1-second synthetic probe encode routine that verifies GPU state prior to scheduling batch jobs, paired with automatic zero-downtime CPU fallback upon GPU driver exhaustion.
   - Enforced strict HLS segmentation conforming to RFC 8216: 4-second chunk duration with exact keyframe alignment (`-g 120 -keyint_min 120 -sc_threshold 0` at 30 fps), enabling seamless multi-bitrate ladder switching.
3. **Embedded Edge Database and Migrations (`internal/adapters/sqlite`):**
   - Implemented the edge database using the pure-Go `modernc.org/sqlite` driver, eliminating external CGO dependencies and enabling static binary distribution.
   - Configured high-throughput database pragmas: Write-Ahead Logging (`WAL` mode), 5,000 ms busy timeout, and memory-backed temporary tables to prevent "database locked" anomalies during concurrent streaming and background transcoding.
   - Authored versioned database migrations tracking media paths, hashes, transcode states, and private watch progress (Invariant 8).
4. **Outbound Reverse Tunnel Client Integration (`internal/adapters/frp`):**

- Implemented the edge-side Fast Reverse Proxy (`frpc`) supervisor daemon, establishing persistent outbound TLS 1.3 connections to the cloud relay server without requiring router port forwarding (Invariant 2).
- Added automated backoff reconnection logic to handle residential broadband IP reassignments and transient network drops.

5. **Standalone Mini-Jellyfin Embedded Offline Web Hub:**
   - Built the self-contained, standalone single-page application (`embedded_ui.go`) served directly by the Go edge daemon on port `8089`.
   - Integrated direct LAN HLS playback and real-time hardware status indicators, enabling complete offline functionality when external cloud connectivity is severed.
6. **Hardware Benchmarking and Multi-PC Lab Demonstration:**
   - Authored the GPU benchmarking script (`scripts/bench-gpu`) and executed comparative tests evaluating encoding throughput (322.3 fps NVENC vs. 33.6 fps CPU) and thermal characteristics.
   - Managed edge deployment on the NVIDIA GeForce RTX 4060 hardware node during cross-network evaluations.

7.3 Contribution of Anosh Chodankar (Seat No.: 25P0320010)

Anosh Chodankar served as the primary architect and lead developer for the **Cloud Control Plane (`apps/api`)**, distributed coordination engines, presentation tier (`apps/web`), and cloud virtualization. His principal contributions include:

1. **Cloud Control API Architecture (`apps/api`):**
   - Designed and developed the high-throughput Go HTTP API utilizing standard library primitives (`net/http`, `log/slog`) adhering to Hexagonal Architecture principles.
   - Implemented versioned REST routing (`/v1/auth/*`, `/v1/nodes/*`, `/v1/catalog`, `/v1/media/*`) with strict schema validation.
2. **PostgreSQL Schema Design and Manifest Synchronization:**
   - Authored PostgreSQL relational schemas for multi-tenant user authentication, edge node registry, and public media catalogs.
   - Implemented connection pooling via `pgxpool` and compile-time type-safe database access routines generated by `sqlc`.
   - Built the manifest synchronization worker that ingests ≤ 3 KB JSON public manifests, updating the global catalog while ensuring video binaries remain exclusively at the edge (Invariant 1).
3. **Distributed Ephemeral Liveness Engine (`internal/adapters/redis`):**

- Engineered the distributed node health tracking subsystem using Redis ephemeral keys with 20-second Time-To-Live (`heartbeat:{node_id}`).
- Implemented automated zero-polling offline detection: node disconnects automatically expire the Redis key, flagging the node as *Offline* across the public catalog within seconds.

4. **Origin Shield Pull-Through Caching and Security (`internal/adapters/s3`):**
   - Designed the `ChunkCache` interface and built the S3-compatible pull-through cache adapter utilizing the AWS SDK for Go v2 (compatible with AWS S3, MinIO, and Floci).
   - Implemented the single-ingest caching logic (Invariant 7): cache misses pull video chunks once through the FRP reverse tunnel, store them in S3 with a 24-hour auto-purge LRU policy, and serve all subsequent concurrent viewers directly from the cache.
   - Implemented cryptographic URL signing using HMAC-SHA256, generating tamper-proof streaming tokens with short-lived expirations to prevent hotlinking and unauthenticated chunk scraping (Invariant 9).
5. **Next.js 15 Presentation Tier and Performance Telemetry (`apps/web`):**
   - Developed the responsive web frontend utilizing Next.js 15 App Router and React Server Components (RSC) for fast server-rendered catalog feeds.
   - Engineered the custom video player using `hls.js`, including adaptive bitrate rendition controls and the engineering telemetry overlay ("Stats for Nerds") displaying live buffer health, bitrate, dropped frames, and cache status.
   - Built the "My Edge Library" management modal and the Developer Diagnostics and Topology Configuration interface.
6. **Cloud Infrastructure and Live Deployment Orchestration:**
   - Configured the live production cloud tier on Google Cloud Platform (GCP `e2-standard-2`), orchestrating Caddy automated Let's Encrypt TLS reverse proxying, FRP server (`frps`), and containerized services.
   - Configured local cloud emulation environments using Floci for offline laboratory demonstration profiles.

7.4 Joint Responsibilities and Shared Milestones

Beyond individual subsystem ownership, both members actively collaborated on foundational system design and integration milestones:

- **Invariants and System Constraints:** Co-authored the 10 Non-Negotiable Invariants (`docs/context/INVARIANTS.md`), establishing project boundaries regarding storage, networking, licensing, and admission limits.
- **Admission Control and Capacity Limiter Design (ADR 0002):** Jointly formulated the 10-stream concurrent remote cap per node (Invariant 10), implementing the dual-path stream router and HTTP 429 Retry-After handling.

- **Concurrency and Load Simulation:** Jointly designed and conducted the automated 10-viewer load simulation script (`scripts/load-10-viewers`), validating Origin Shield single-ingest bandwidth protection.
- **Documentation and Academic Deliverables:** Co-authored technical runbooks (`DEMO_ALL_MODES.md`, `RUNNING_GUIDE.md`), the project presentation deck, and this comprehensive academic project report.

7.5 Summary Contribution Matrix

Table 7.1 summarizes the quantitative module ownership, specific deliverables, and relative workload distribution between team members.

Table 7.1: System Subsystem Responsibility and Workload Contribution Matrix

Subsystem / Component	**Primary Deliverables**	**Primary Owner**	**Anish**	**Anosh**
Edge Ingestion & Hasher	Recursive Scanner, SHA-256 Hasher	Anish Ghadi	**80%**	20%
Transcoder & NVENC	Subprocess FFmpeg, NVENC Probe, Fallback	Anish Ghadi	**85%**	15%
Embedded Edge Database	SQLite Engine, WAL Mode, Migrations	Anish Ghadi	**85%**	15%
Reverse Tunnel (Edge)	FRP Client Daemon, TLS 1.3 Multiplex	Anish Ghadi	**75%**	25%
Mini-Jellyfin Offline Hub	Standalone Embedded HTML5/JS Web UI	Anish Ghadi	**80%**	20%
Hardware Benchmarking	NVENC vs. CPU Benchmark Script	Anish Ghadi	**85%**	15%
Cloud Control API	Go `net/http` REST API, `log/slog`	Anosh Chodankar	20%	**80%**
PostgreSQL Relational DB	Schema Design, `pgx`, `sqlc` Queries	Anosh Chodankar	15%	**85%**
Redis Liveness Tracking	Ephemeral Heartbeat TTL Keys	Anosh Chodankar	20%	**80%**
Origin Shield Cache	S3 Adapter, 24h LRU Purge, Single Ingest	Anosh Chodankar	25%	**75%**
HMAC Token Signer	Playback URL Signing, Expiry Validation	Anosh Chodankar	20%	**80%**
Next.js Presentation Tier	App Router, Player, Stats HUD, Modals	Anosh Chodankar	15%	**85%**
Cloud Deployment	GCP VM, Caddy Auto-TLS, Floci Setup	Anosh Chodankar	20%	**80%**
Contracts & Architecture	10 Invariants, JSON Schemas, OpenAPI	Joint	**50%**	**50%**
Admission Limiter	Stream Router, 10-Stream Cap, HTTP 429	Joint	**50%**	**50%**
Load Simulation	10-Viewer Automated Stress Testing	Joint	**50%**	**50%**
Report & Presentation	Academic LaTeX Report, Slide Deck	Joint	**50%**	**50%**
Aggregate Project Workload	**Full Distributed System Delivery**	**Equal Effort**	**50%**	**50%**

CHAPTER 8

Conclusion and Future Scope

The exponential growth of digital multimedia has historically concentrated control within centralized hyperscale cloud providers, burdening creators and institutions with severe recurring storage and egress liabilities. This project, **CustardFin**, demonstrates that by strategically partitioning computational and storage responsibilities between consumer edge hardware and a lightweight cloud control plane, an open-source video-on-demand and community sharing platform can operate with high reliability, absolute data sovereignty, and near-zero cloud operating costs.

8.1 Summary of Achievements

CustardFin successfully fulfills all core functional requirements (FR-1 through FR-10) established in the Business Requirements Document and rigorously upholds all ten non-negotiable architectural invariants:

- **Decoupled Zero-Storage Architecture (Invariant 1):** Master video binaries remain permanently on consumer-owned edge drives. The cloud tier persists only lightweight 650-byte JSON metadata manifests, eliminating perpetual cloud object storage liabilities (100% storage cost reduction).
- **Residential CGNAT Traversal (Invariant 2):** Home edge nodes connect to the cloud control plane exclusively via outbound TLS 1.3 reverse tunnels (FRP), eliminating the need for open inbound router ports, UPnP, or static public IPv4 allocations.
- **100% Free and Open-Source Stack (Invariant 3):** Constructed entirely upon permissively licensed open-source software (Go, PostgreSQL, Redis, MinIO/Floci, FRP, Next.js, and SQLite), completely avoiding proprietary licenses or vendor lock-in.
- **High-Performance Hardware Transcoding (Invariant 6):** Pre-encoded HLS packaging with strict 4-second Group of Pictures keyframe alignment (`-g 120`) leverages dedicated NVIDIA NVENC ASICs, delivering 10.74× real-time encoding throughput (322 + fps) while maintaining host CPU utilization under 5%, backed by automated CPU fallback.

- **Origin Shield Bandwidth Protection (Invariant 7):** The ephemeral 24-hour pull-through cache guarantees single-ingest protection across residential broadband uplinks. During automated 10-viewer load simulations, the home node uploaded each hot segment exactly once, offloading 90% of streaming bandwidth to the cache and sustaining high-definition playback on constrained 20 Mbps residential lines.
- **Absolute Private Data Sovereignty (Invariant 8):** User viewing histories, playback bookmarks, and unshared personal recordings remain strictly within the edge node's local SQLite database.
- **Cryptographic Token Access Control (Invariant 9):** Streaming segments are guarded by short-lived HMAC-SHA256 signed URLs, preventing unauthenticated scraping and cross-site hotlinking.
- **Admission Control and Capacity Enforcement (Invariant 10):** The dual-path stream router strictly enforces a 10-stream concurrent remote cap per node, returning graceful `HTTP 429 Too Many Requests` responses to protect residential networks from upload exhaustion.
- **Sub-5ms Local LAN Playback:** Proximity-resolved dual-path streaming and the embedded Mini-Jellyfin offline web hub enable direct local network streaming with under 4 ms TTFB, providing full media streaming even during total cloud or ISP outages.
- **Confirmed 95.89% Cloud Cost Reduction:** Financial analysis established a monthly operating cost of ₹ 4,128.60 ($49.15) compared to ₹ 1,00,380.00 ($1,195.00) for conventional centralized cloud architectures, yielding verified savings of over ₹ 34.65 Lakhs ($41,250) across a 3-year lifecycle for a 100-user community.

8.2 Limitations of the Current System (Version 1)

While CustardFin achieves production-grade functionality for community and self-hosted environments, several architectural boundaries are acknowledged:

1. **Centralized Relay VM Dependency:** Although video binaries are not stored in the cloud, remote tunnel traffic for cache-miss chunks transits a single cloud relay instance running `frps`. If this relay virtual machine suffers network partition, remote playback for uncached chunks stalls until connectivity recovers.
2. **Absence of Edge-to-Edge P2P Swarming:** In Version 1, edge nodes communicate exclusively with the cloud control plane. Direct browser-to-browser WebRTC DataChannel chunk sharing is not yet enabled.
3. **Lack of Hardware Digital Rights Management (DRM):** To preserve open-source software purity and avoid proprietary licensing fees, CustardFin relies on HMAC-SHA256 signed URLs and dynamic HLS chunking rather than hardware-enforced DRM schemes (e.g., Google Widevine Modular or Apple FairPlay).
4. **Browser-Centric Client Experience:** The client interface is currently deployed as a responsive web application for desktop and mobile browsers. Dedicated native applications for smart television platforms (Android TV, Apple tvOS, Roku) are not yet packaged.

8.3 Future Research and Engineering Enhancements

The modular, hexagonal design of CustardFin provides a robust foundation for future enhancements:

- **Multi-Region Anycast FRP Relays:** Future iterations can deploy geographically dispersed FRP relay servers operating behind BGP Anycast and GeoDNS. Edge nodes will automatically bind to the topologically nearest relay, minimizing global round-trip latency and eliminating single-relay points of failure.
- **Hybrid WebRTC Peer-Assisted Delivery:** For popular community streams experiencing sudden viewership surges, the Next.js web player can be extended with client-side WebTorrent / WebRTC DataChannel protocols. Concurrent viewers can trade cached HLS segments directly, further reducing cloud egress and relay bandwidth.
- **Cryptographic Manifest Attestation and Decentralized Moderation:** Manifest exchanges can incorporate public-key cryptography (Ed25519) to verify node ownership and metadata integrity. Community moderation mechanisms (such as web-of-trust reputation scoring) can empower user collectives to curate public catalogs autonomously.
- **Cross-Platform Native Smart TV Applications:** Leveraging the standardized, versioned OpenAPI 3.0 specification (`docs/api/openapi.yaml`), native television interfaces built in Flutter or React Native can bring the CustardFin experience to living room displays with seamless direct LAN auto-discovery.

8.4 Concluding Remarks

CustardFin resolves the historic tension between personal media privacy and community content discoverability. By demonstrating that video binaries can remain securely at home while a centralized cloud plane orchestrates discovery, reverse tunneling, and temporary caching, the project establishes a sustainable paradigm for the future of decentralized multimedia systems. This project fulfills the rigorous academic and practical requirements of the MCA Cloud Computing Laboratory curriculum, presenting a fully realized, empirically validated, and economically revolutionary distributed system.

Bibliography

[1] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[2] Amazon Web Services, Inc., "Amazon S3 Pricing: Cloud Object Storage Pricing," https://aws.amazon.com/s3/pricing/, 2024.

[3] Jellyfin Project Contributors, "The Free Software Media System," https://jellyfin.org/, 2024.

[4] J. Weil, V. Kuarsingh, C. Donley, C. Liljenstolpe, and M. Azinger, "IANA-Reserved IPv4 Prefix for Shared Address Space," Internet Engineering Task Force (IETF), RFC 6598, Apr. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6598

[5] F. Lemmet and Framasoft, "PeerTube: A free, decentralized and federated video platform powered by ActivityPub and WebTorrent," https://joinpeertube.org/, 2021.

[6] C. Lemus, E. Prodromou, K. Guy, and T. Tallon, "ActivityPub: W3C Recommendation," World Wide Web Consortium (W3C), Tech. Rep., Jan. 2018. [Online]. Available: https://www.w3.org/TR/activitypub/

[7] B. Cohen, "Incentives build robustness in BitTorrent," *Workshop on Economics of Peer-to-Peer Systems*, vol. 6, pp. 68–72, 2003.

[8] T. Wiegand, G. J. Sullivan, G. Bjontegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, 2003.

[9] NVIDIA Corporation, *NVIDIA Video Codec SDK: NVENC Programming Guide*, NVIDIA Developer Documentation, 2023, version 12.1.

[10] R. Pantos and W. May, "HTTP Live Streaming (HLS)," Internet Engineering Task Force (IETF), RFC 8216, Aug. 2017. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8216

[11] fatedier, "frp: A fast reverse proxy to help you expose a local server behind a NAT or firewall to the internet," https://github.com/fatedier/frp, 2024.

[12] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," Internet Engineering Task Force (IETF), RFC 8446, Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446